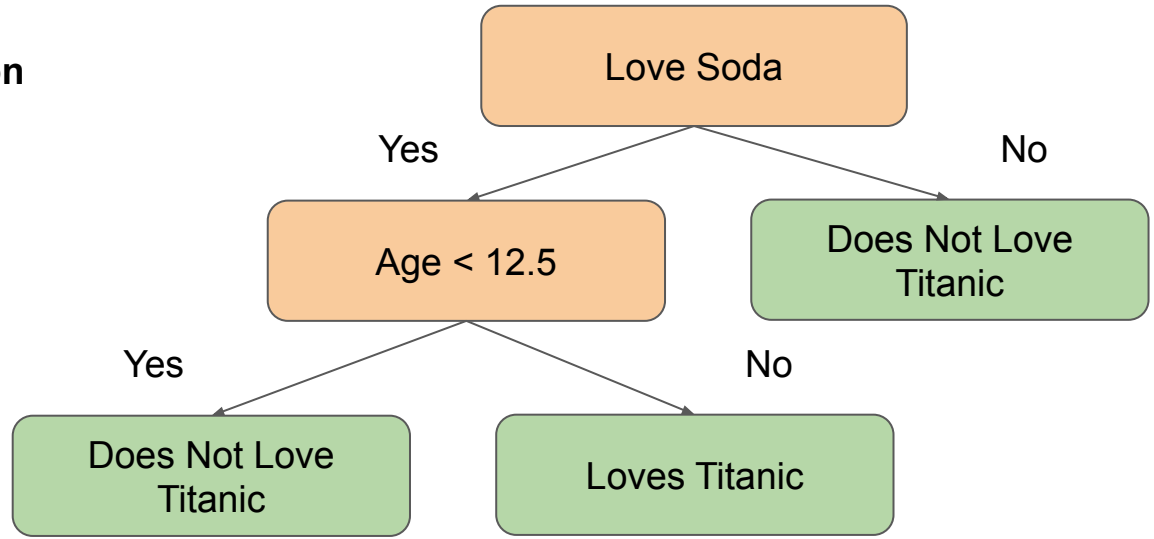
Decision Trees

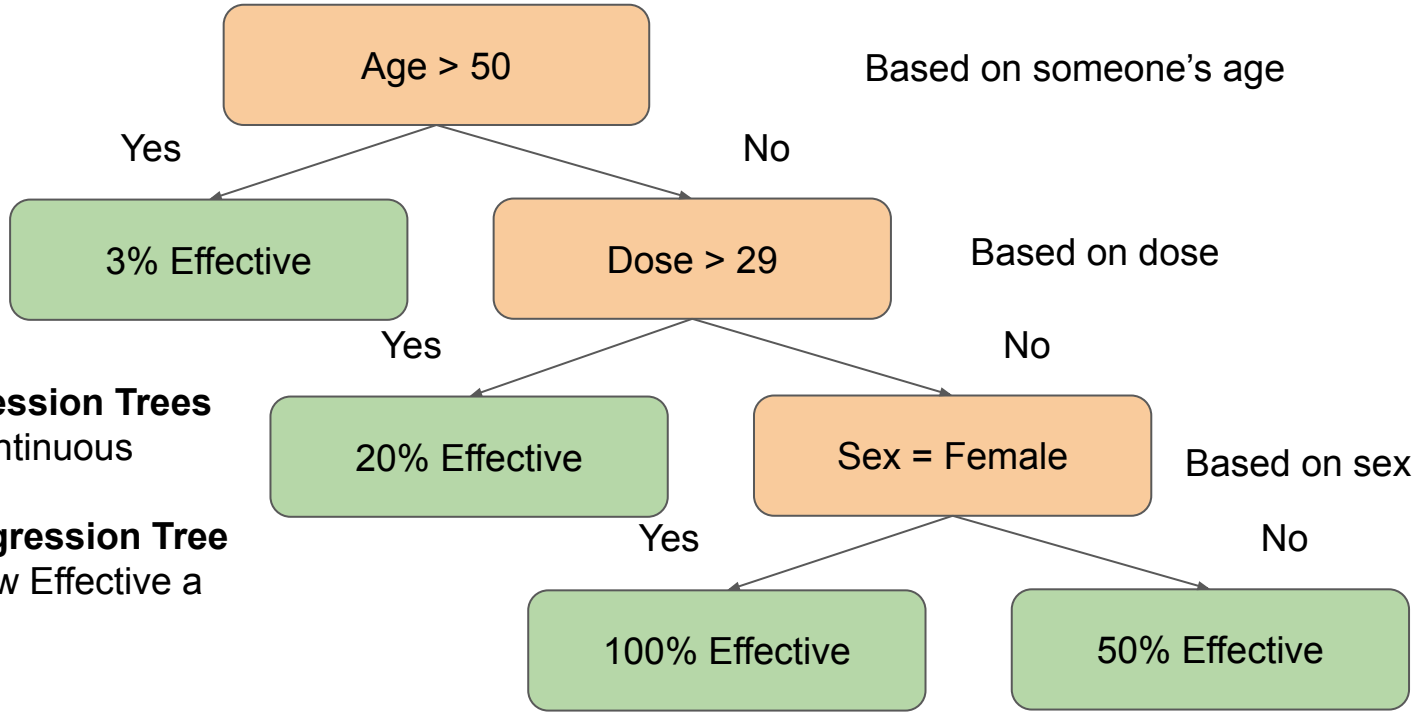
Lecturer: Albert Yang (楊景明)

Classification and Regression Trees

- There are two types of **Trees** in machine learning: **Classification** and **Regression**
- **Classification Trees** classify people or things into two or more discrete categories



Classification and Regression Trees



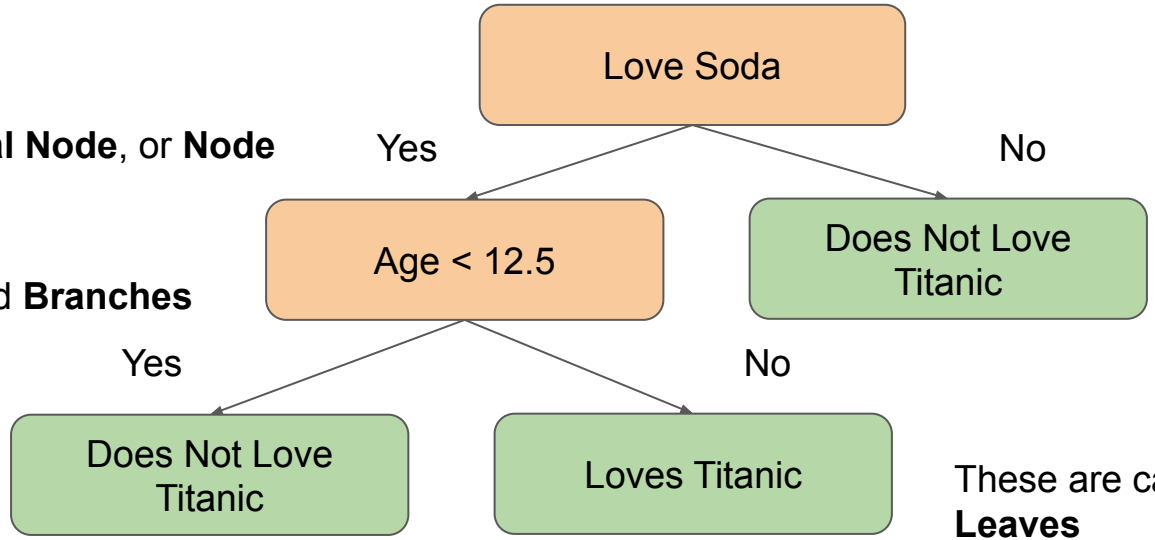
- In contrast, **Regression Trees** try to predict a continuous value.
- This example **Regression Tree** tries to predict how Effective a drug will be

Terminology

The top of the tree is called **Root Node**, or **Root**

This is called **Internal Node**, or **Node**

The arrows are called **Branches**



These are called **Leaf Node**, or **Leaves**

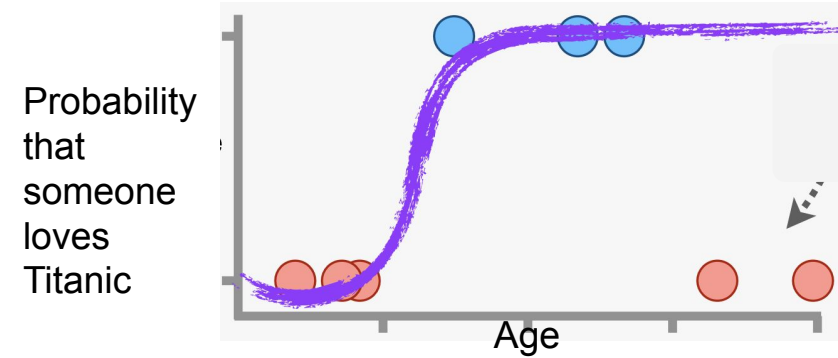
Classification Trees

Example

We want to predict if someone loves Titanic

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

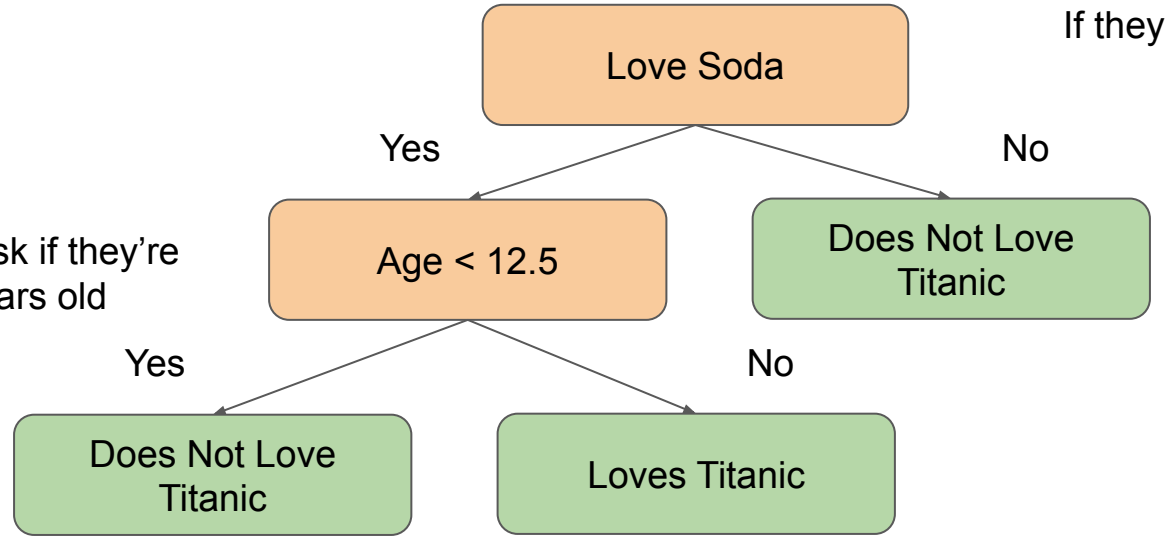
- A standard **Logistic Regression** assumes the probability only rises or falls with Age, so it misses this up-and-down pattern.
- This plot shows Age only for illustration



Solution

- A **Classification Tree** is one option. It works with both yes/no and numeric features, and it finds cut points from the data. This lets it capture curved patterns and interactions without us specifying them in advance. The trade-off is that a single tree can overfit and may change a lot with small changes in the data.

- **Classification Trees** are relatively easy to interpret and use
- If you meet a new person and want to decide if they love Titanic or not, you simply start at the top and ask if they love Soda



If they love Soda?

If they do, then ask if they're less than 12.5 years old

And if they don't love Soda to begin with, they probably do not love Titanic

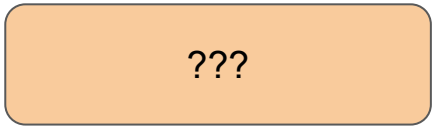
If they are, then they probably do not love Titanic

If not, then they probably love Titanic

Building a Classification Tree

The first thing we do is decide whether Loves Popcorn, Loves Soda, or Age should be the question we ask at the very top of the tree.

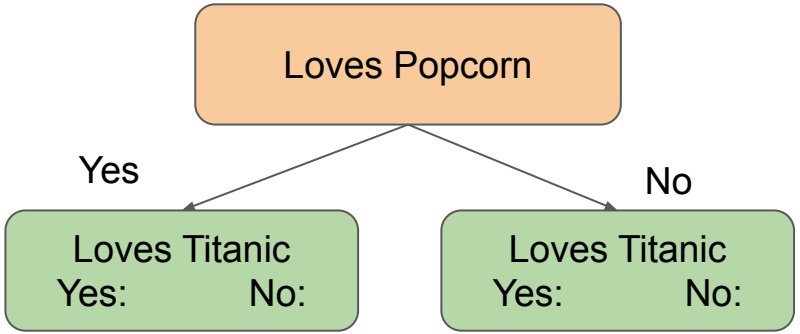
Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Start with Loves Popcorn

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

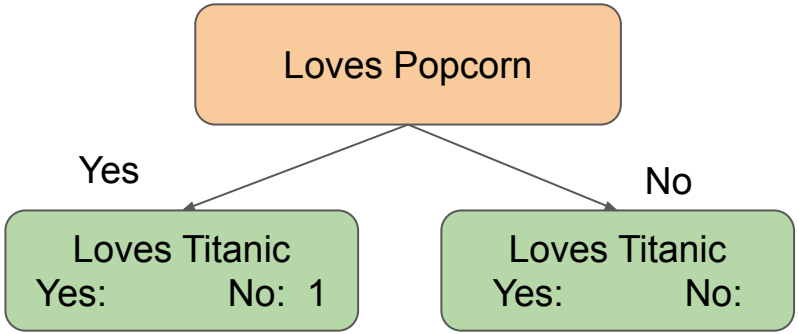
- We'll start by looking at how well Loves Popcorn predicts whether or not someone loves Titanic by making a super simple tree that only asks if someone loves Popcorn



Start with Loves Popcorn

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

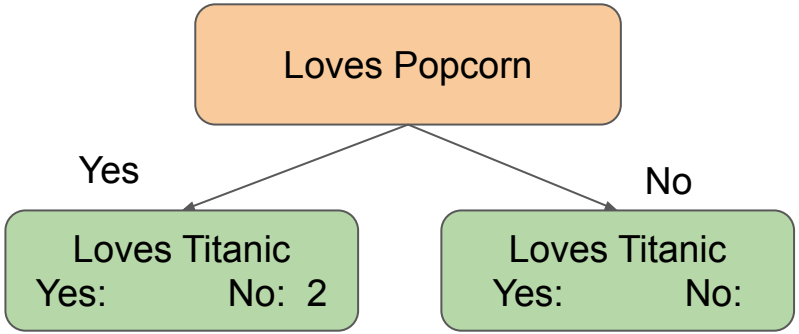
- For exampleThe first person in the Training Data loves Popcorn, so they go to the Leaf on the left, and because they do not love Titanic, we'll put a 1 under the word



Start with Loves Popcorn

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

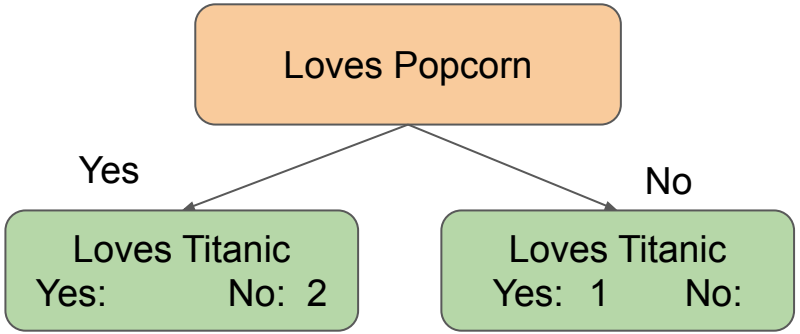
- The second person also loves Popcorn, so they also go to the Leaf on the left, and because they do not love Titanic, we increase No to 2



Start with Loves Popcorn

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

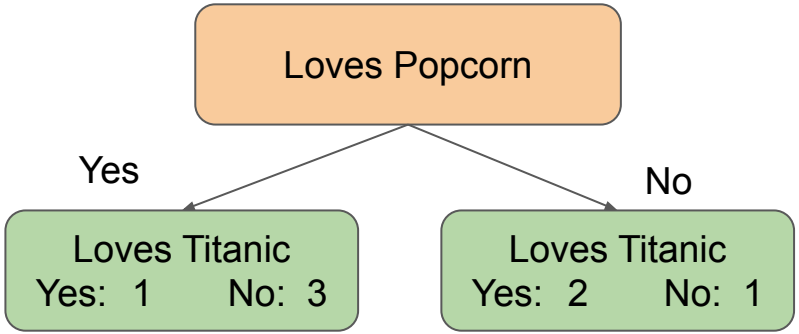
The third person does not love Popcorn, so they go to the Leaf on the right, but they love Titanic, so we put a 1 under the word Yes



Start with Loves Popcorn

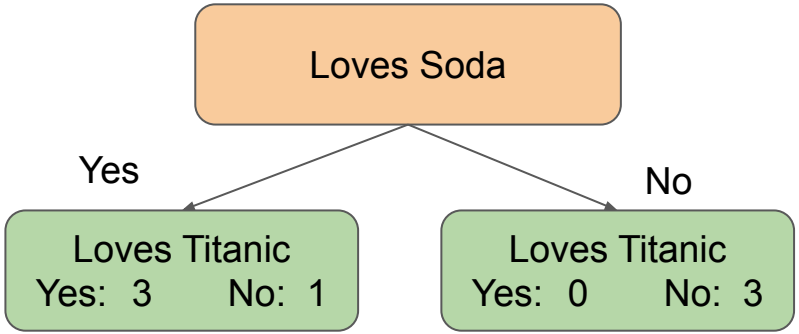
We run the remaining rows down the tree, keeping track of whether or not each person loves Titanic or not.

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

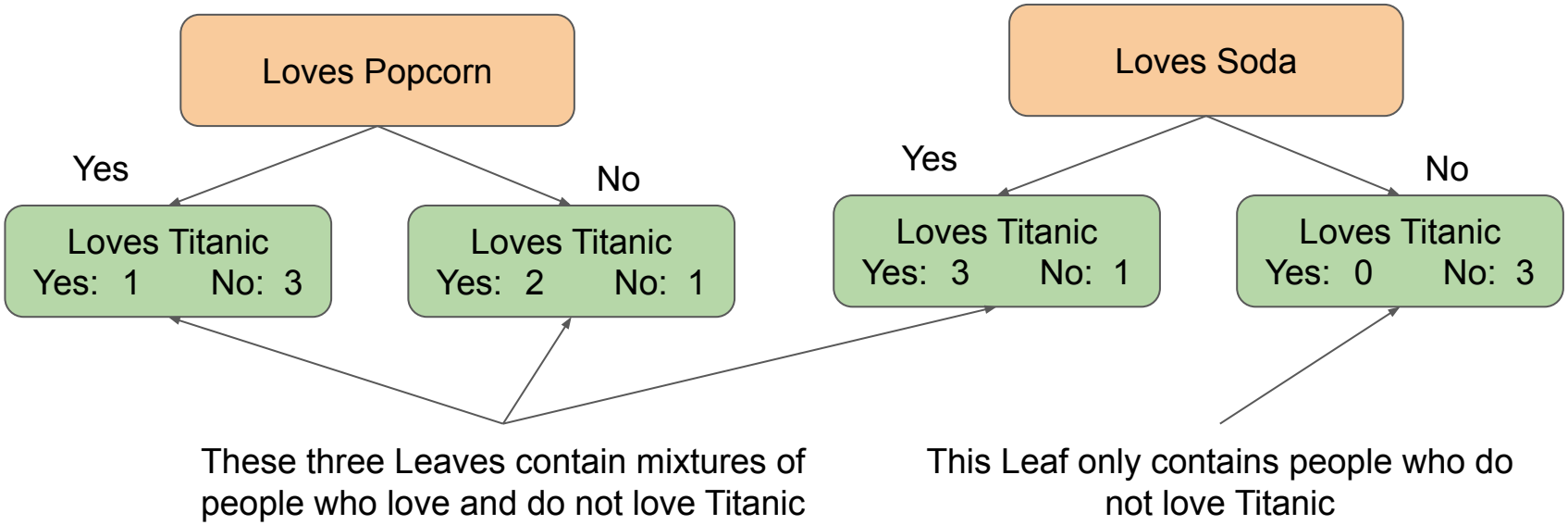


Do the Same Thing for Loves Soda

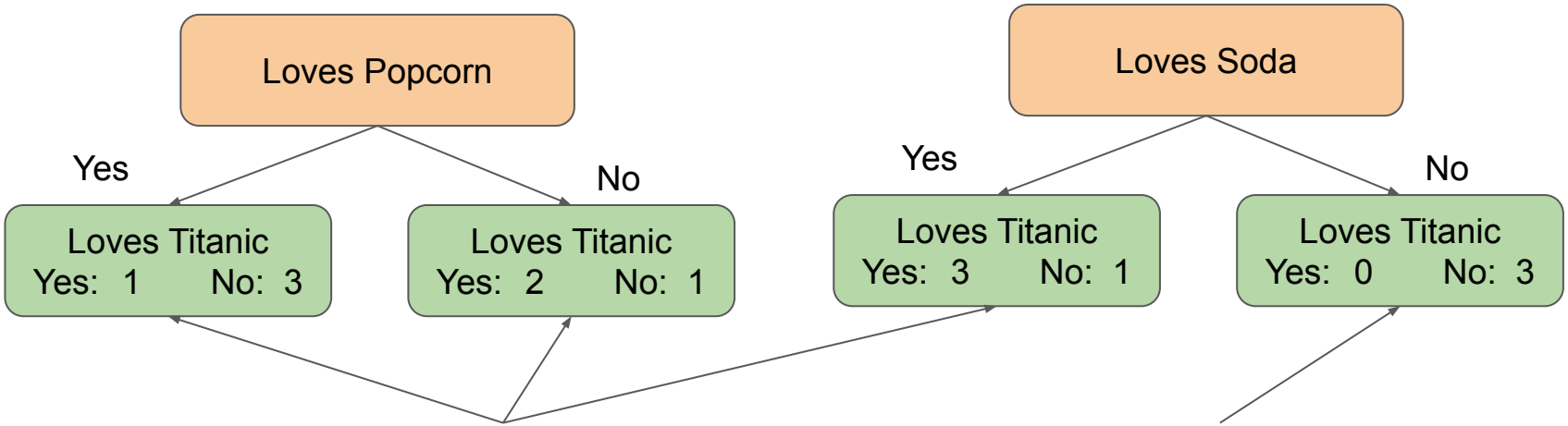
Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Compare the Two Trees

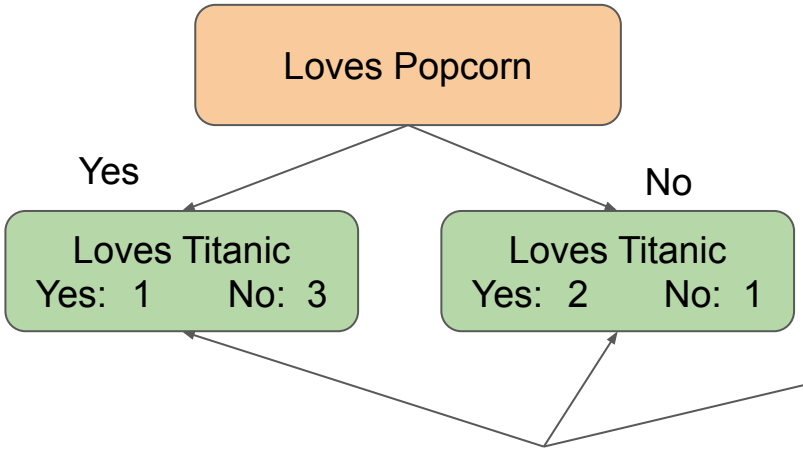


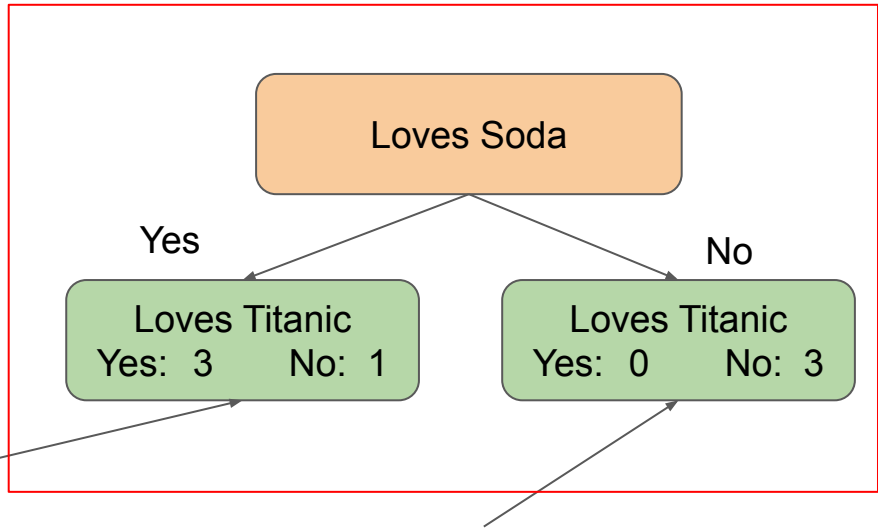
Compare the Two Trees



Compare the Two Trees

It seems like Loves Soda does a better job classifying who loves and does not love Titanic





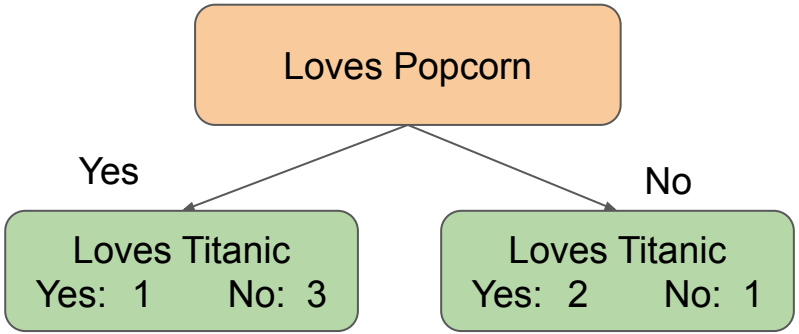
These three Leaves contain mixtures of people who love and do not love Titanic

This Leaf only contains people who do not love Titanic

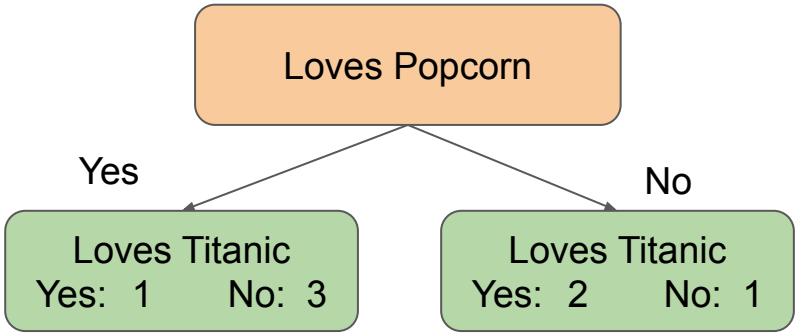
Leaves that contain mixtures of classifications are called **Impure**

Quantify the Impurity of Leaves and Trees

- **Gini Impurity**
- **Entropy**
- **Information Gain**

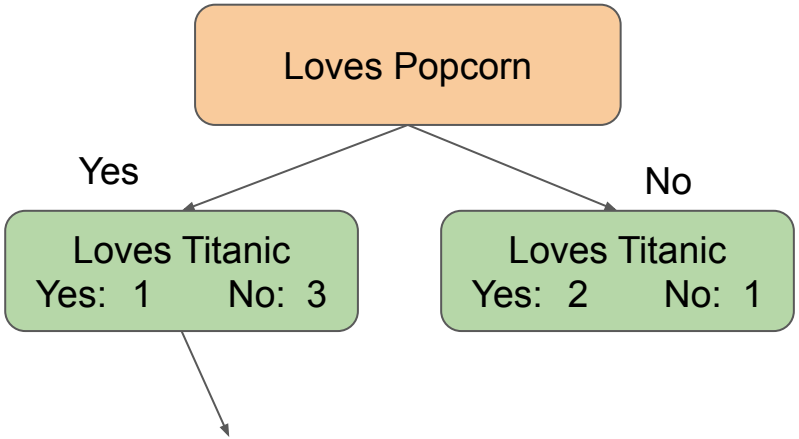


Gini Impurity for Loves Popcorn



Gini Impurity for a leaf = 1 - (the probability of “Yes”)^2 - (the probability of “No”)^2

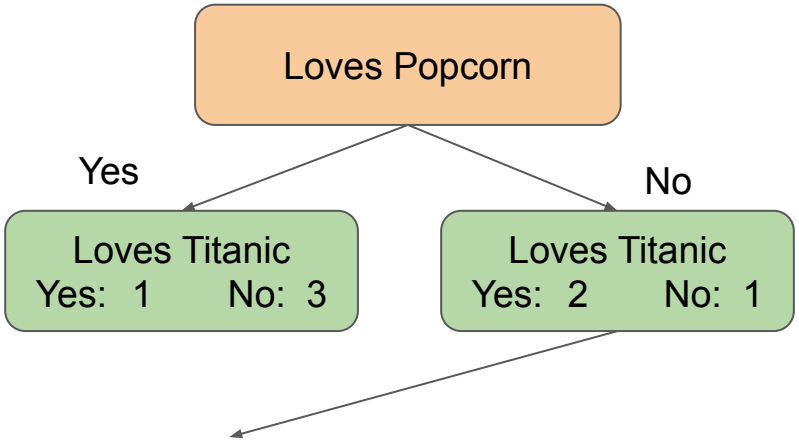
Gini Impurity for Loves Popcorn



Gini Impurity for the leaf on the left = 1 - (the probability of "Yes")^2 - (the probability of "No")^2

= 1 - $(1/4)^2$ - $(3/4)^2$ = **0.375**

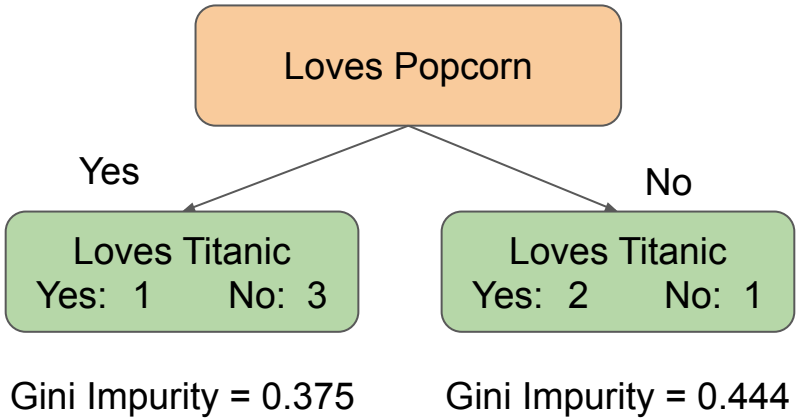
Gini Impurity for Loves Popcorn



Gini Impurity for the leaf on the right = 1 - (the probability of "Yes")^2 - (the probability of "No")^2

= 1 - $(2/3)^2$ - $(1/3)^2$ = **0.444**

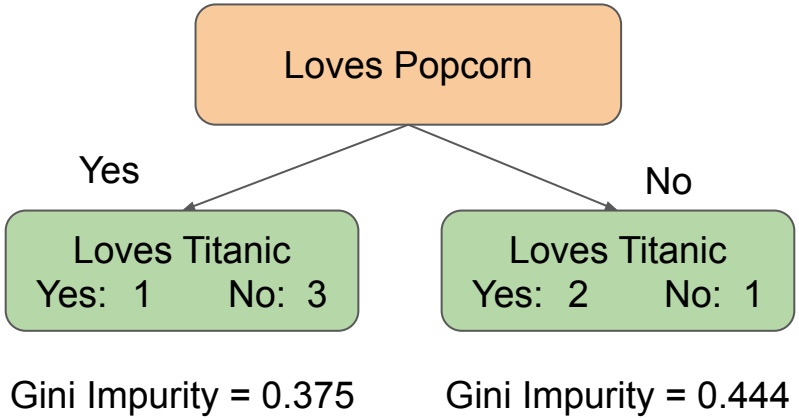
Quantify the Impurity of Leaves and Trees



Total **Gini Impurity** = weighted average of Gini Impurities for the Leaves

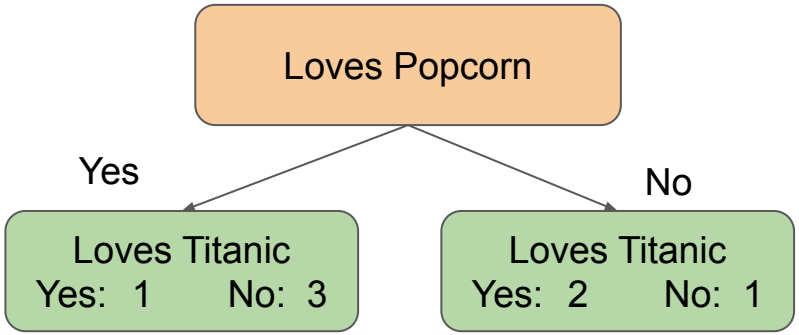
- Because the **Leaf** on the left has **4** people in it, and the **Leaf** on the right only has **3**, the **Leaves** do not represent the same number of people
- To compensate for the differences in the number of people in each **Leaf**, the total **Gini Impurity** for Loves Popcorn is the **Weighted Average** of the two **Leaf** Impurities.

Quantify the Impurity of Leaves and Trees

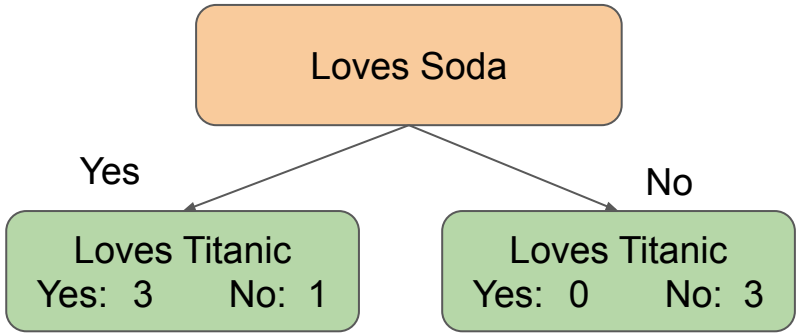


Total **Gini Impurity** = weighted average of Gini Impurities for the Leaves
= (4/7) * 0.375 + (3/7) * 0.444 = 0.405

Compare the Two Trees



Total Gini Impurity = 0.405



Total Gini Impurity = 0.214

The lower Gini Impurity for Loves Soda confirms what we suspected earlier

Calculate the Gini Impurity for Age

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Calculate the Gini Impurity for Age

Normally, the first thing we do is sort the rows by Age, from lowest to highest

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Age	Loves Titanic
7	No
12	No
18	Yes
35	Yes
38	Yes
50	No
83	No

Calculate the Gini Impurity for Age

The next thing we do is calculate the average Age for all adjacent rows

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

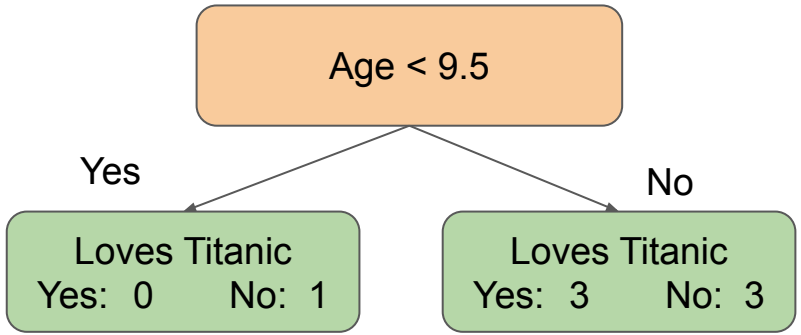
	Age	Loves Titanic
	7	No
9.5		
	12	No
15		
	18	Yes
26.5		
	35	Yes
36.5		
	38	Yes
44		
	50	No
66.5		
	83	No

Calculate the Gini Impurity for Age

Lastly, we calculate the Gini Impurity values for each average Age

	Age	Loves Titanic
	7	No
9.5		
	12	No
15		
	18	Yes
26.5		
	35	Yes
36.5		
	38	Yes
44		
	50	No
66.5		
	83	No

For example, the first average Age is 9.5, so we use 9.5 as the threshold for splitting the rows into 2 leaves



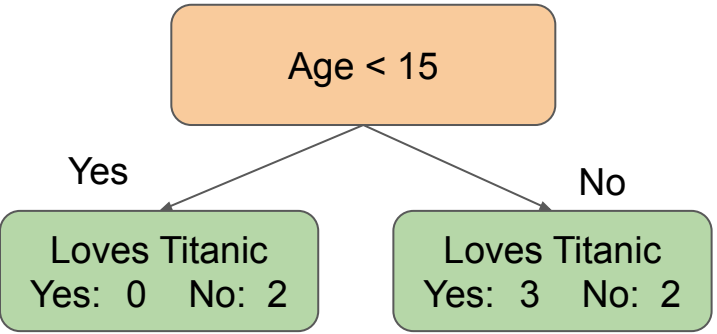
Gini Impurity = 0

Gini Impurity = 0.5

Total **Gini Impurity** = (1/7) * 0 + (6/7) * 0.5 = 0.429

Calculate the Gini Impurity for Age

- Ultimately, we end up with a Gini Impurity for each potential threshold for Age
- We identify the thresholds with the lowest Impurities



Gini Impurity = 0.429 ← 9.5

Gini Impurity = 0.343 ← 15

Gini Impurity = 0.476 ← 26.5

Gini Impurity = 0.476 ← 36.5

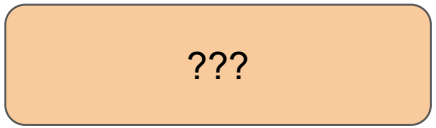
Gini Impurity = 0.343 ← 44

Gini Impurity = 0.429 ← 66.5

Age	Loves Titanic
7	No
12	No
18	Yes
35	Yes
38	Yes
50	No
83	No

Determine the Root

Our first goal was to determine whether we should ask about Loves Popcorn, Loves Soda, or Age at the very top of the tree

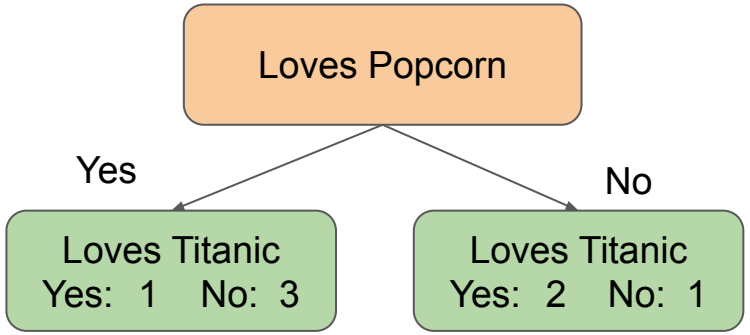


Determine the Root

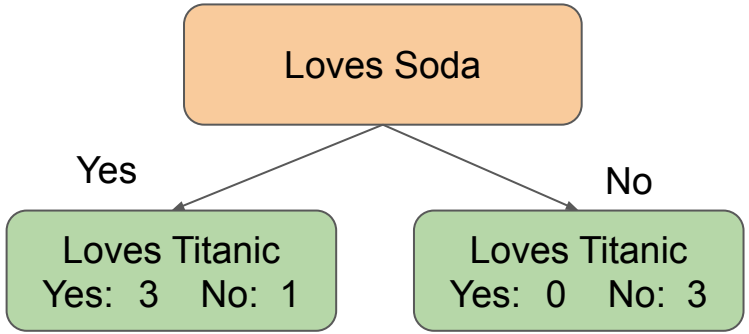
Our first goal was to determine whether we should ask about Loves Popcorn, Loves Soda, or Age at the very top of the tree

???

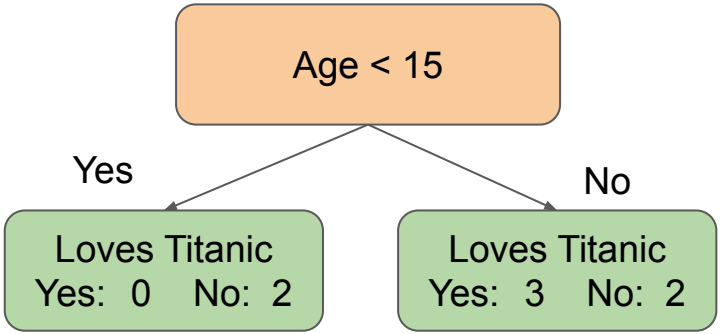
Gini Impurity = 0.405



Gini Impurity = 0.214



Gini Impurity = 0.343

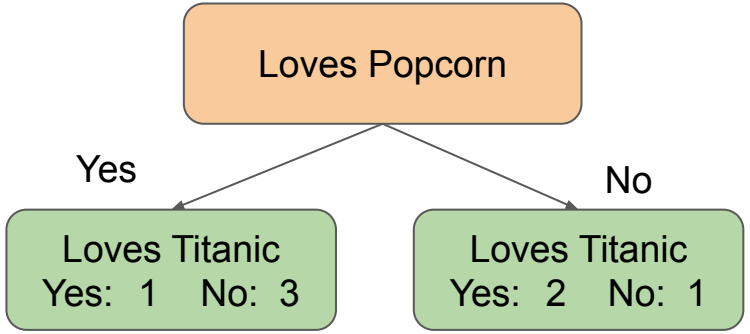


Determine the Root

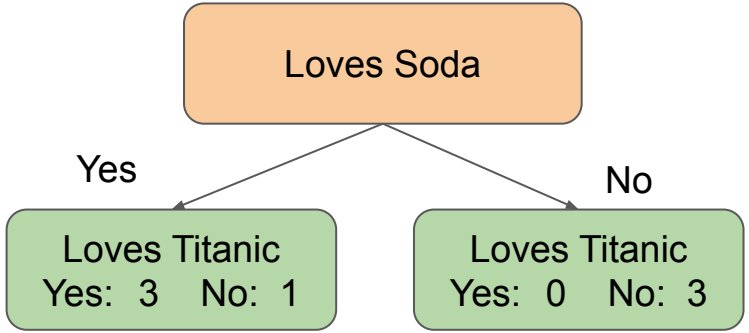
Our first goal was to determine whether we should ask about Loves Popcorn, Loves Soda, or Age at the very top of the tree

Loves Soda

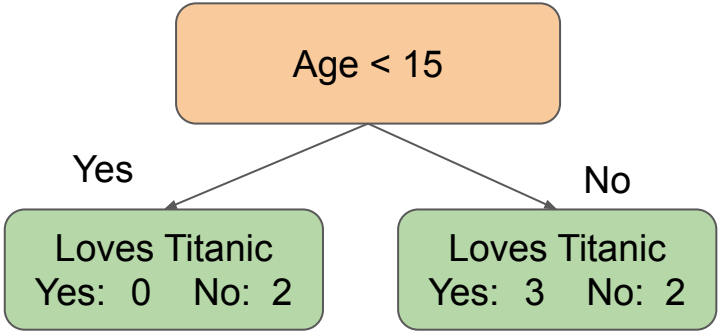
Gini Impurity = 0.405



Gini Impurity = 0.214

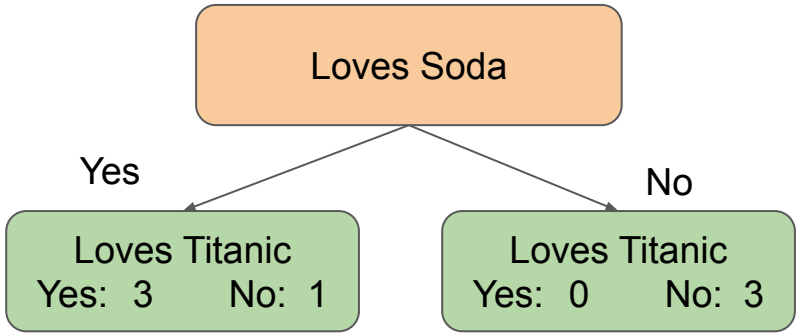


Gini Impurity = 0.343



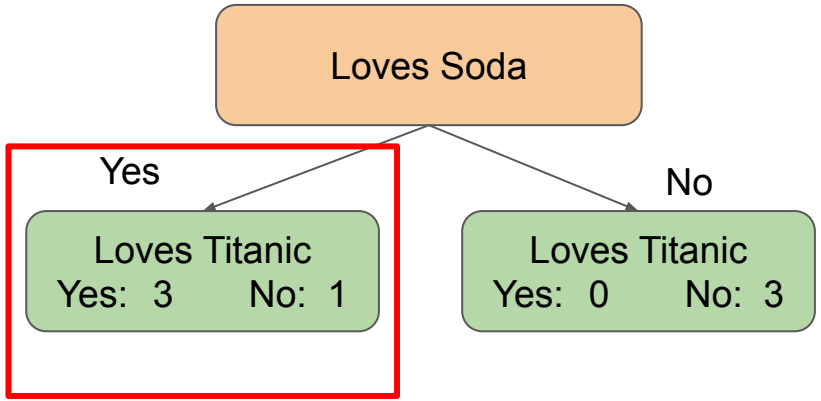
- With Loves Soda at the top of the tree, the **4** people who love Soda, including **3** who love Titanic and **1** who does not, go to the **Node** on the left and the **3** people who do not love Soda, all of whom do not love Titanic, go to the **Node** on the right

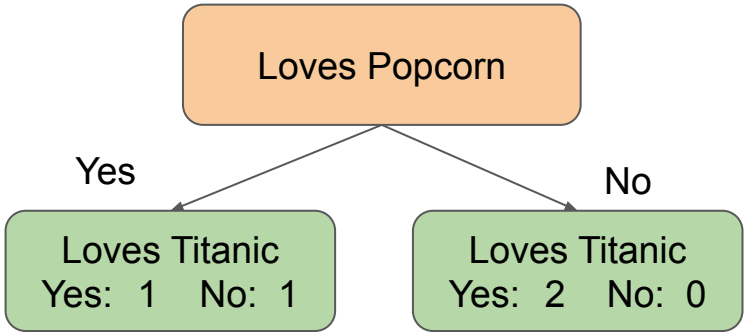
Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



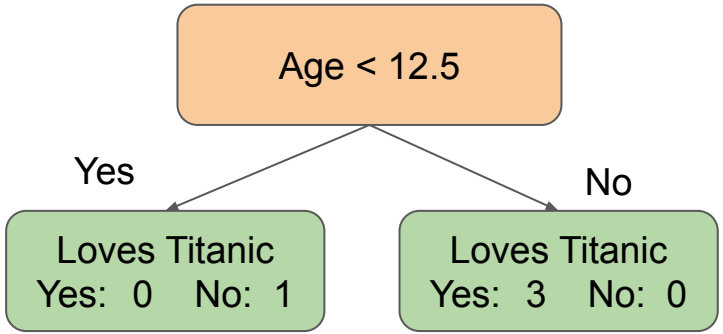
- Now, because the Node on the left is Impure, we can split the 4 people in it based on Loves Popcorn or Age and calculate the Gini Impurities

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No





Gini Impurity = 0.25

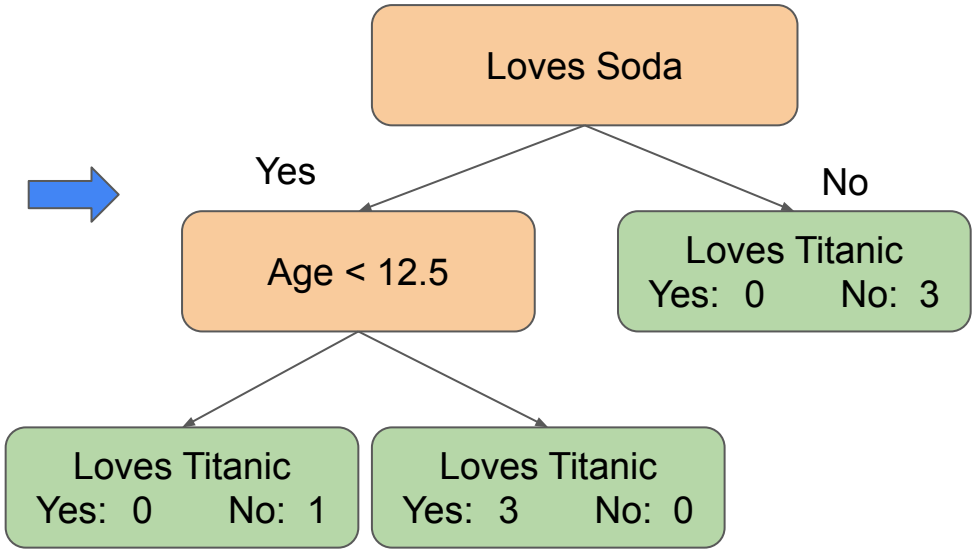


Gini Impurity = 0

- When we split the 4 people who love Soda based on whether or not they love Popcorn, the Gini Impurity is 0.25
- However, when we split the 4 people based on Age < 12.5, the Gini Impurity is 0

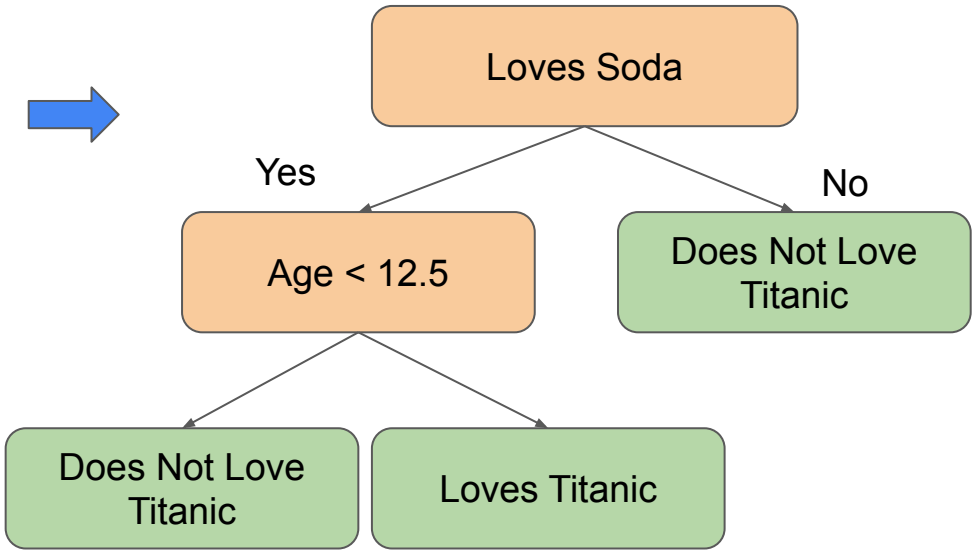
Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

At this point, we've created a Tree from the Training Data

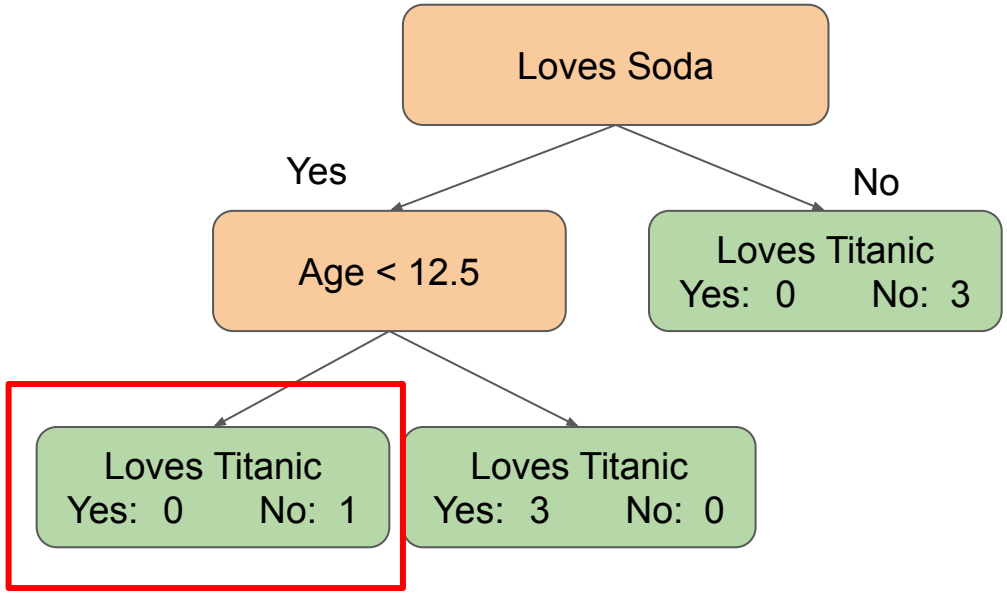


- Generally speaking, the output of a **Leaf** is whatever category that has the most counts

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



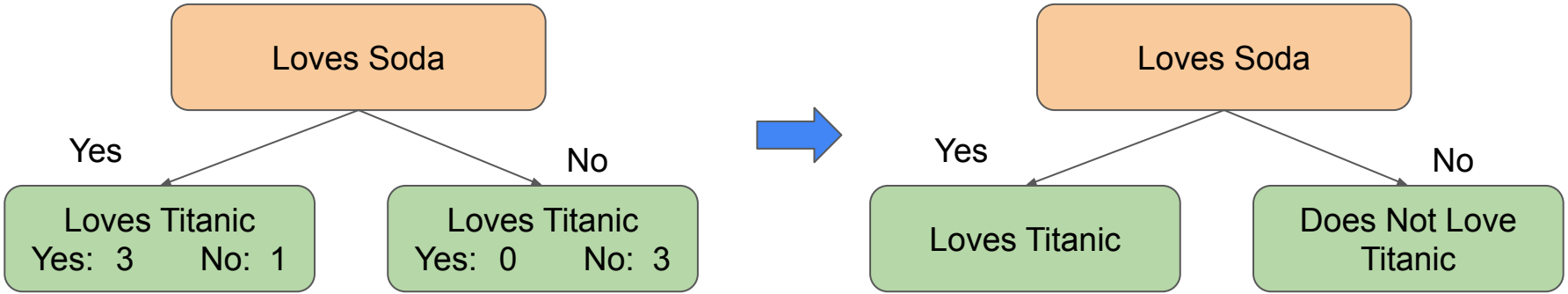
Let's Check the Leaf



- Pruning
- Limit the number of instance in a leaf

It's hard to have confidence that the tree will do a great job making predictions with future data

Require 3 or more people per Leaf

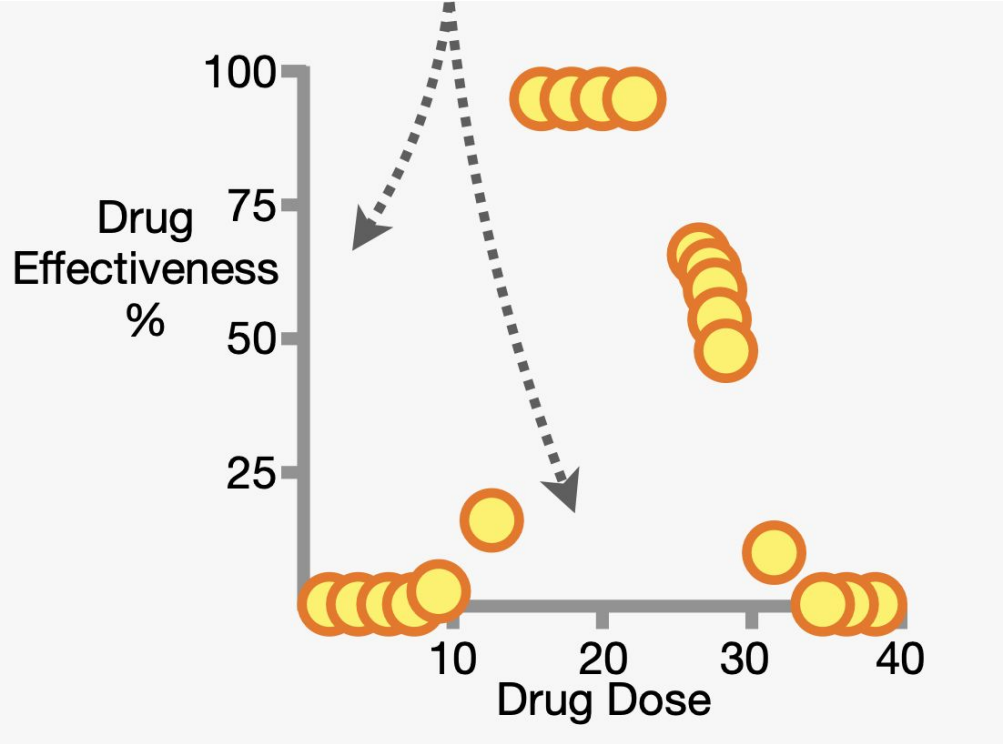


- We can put limits on how trees grow, for example, by requiring **3** or more people per **Leaf**
- The **Leaf** would be **Impure** but we would also have a better sense of the accuracy of our prediction because we know that only **75%** of the people in the Leaf love Titanic.
- We don't know in advance if it's better to require **3** people per **Leaf** or some other number, so we try a bunch, use **Cross Validation**, and pick the number that works best.
- Even though this **Leaf** is Impure, it still needs an output value, and because most of the people in this **Leaf** love Titanic, that will be the output value.

Regression Trees

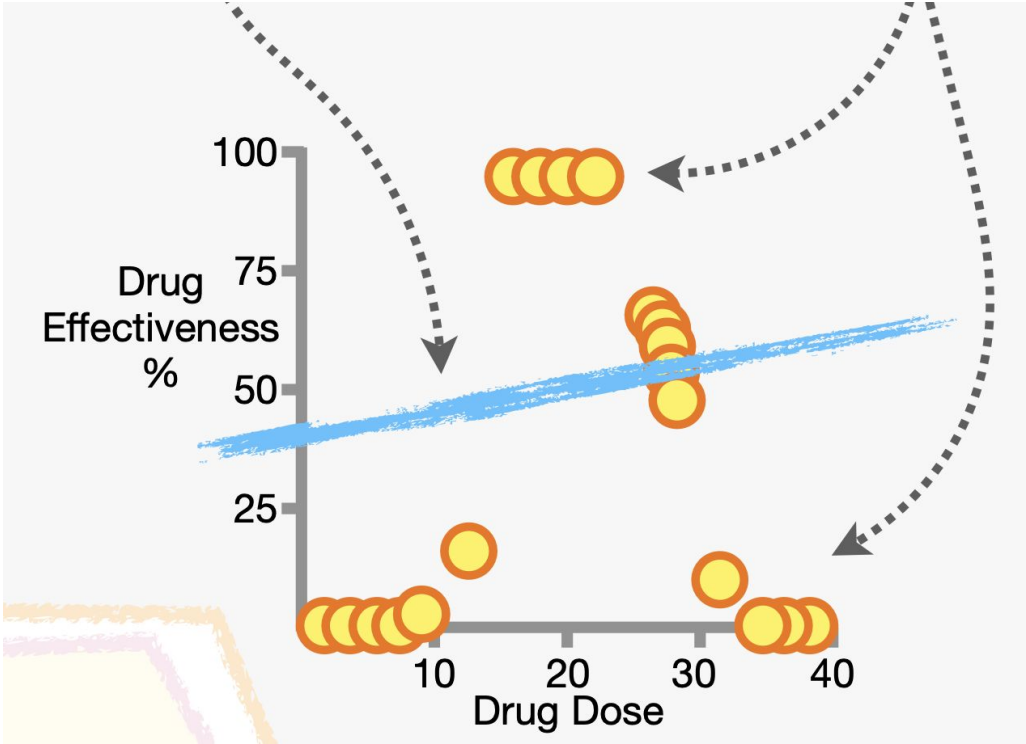
Problem

- We have this Training Dataset that consists of Drug Effectiveness for different Doses

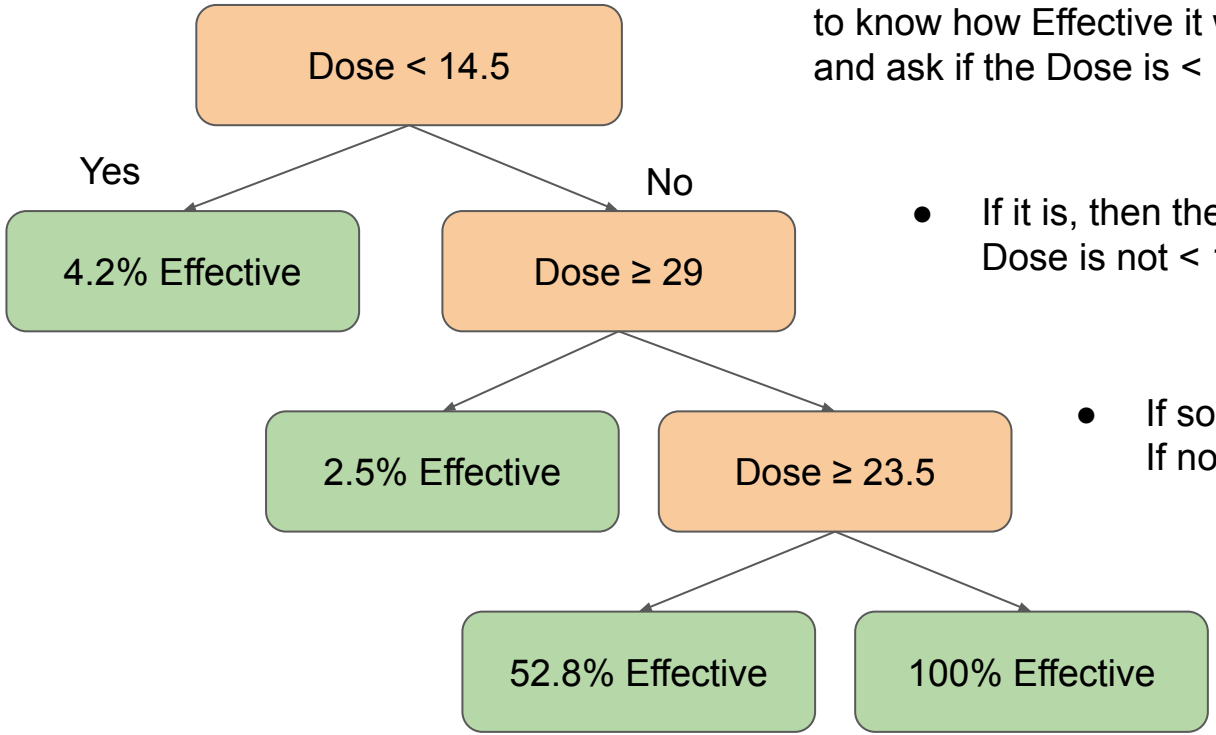


If We Fit a Line…

- Fitting a straight line to the data would result in terrible predictions because there are clusters of ineffective Doses that surround the effective Doses.

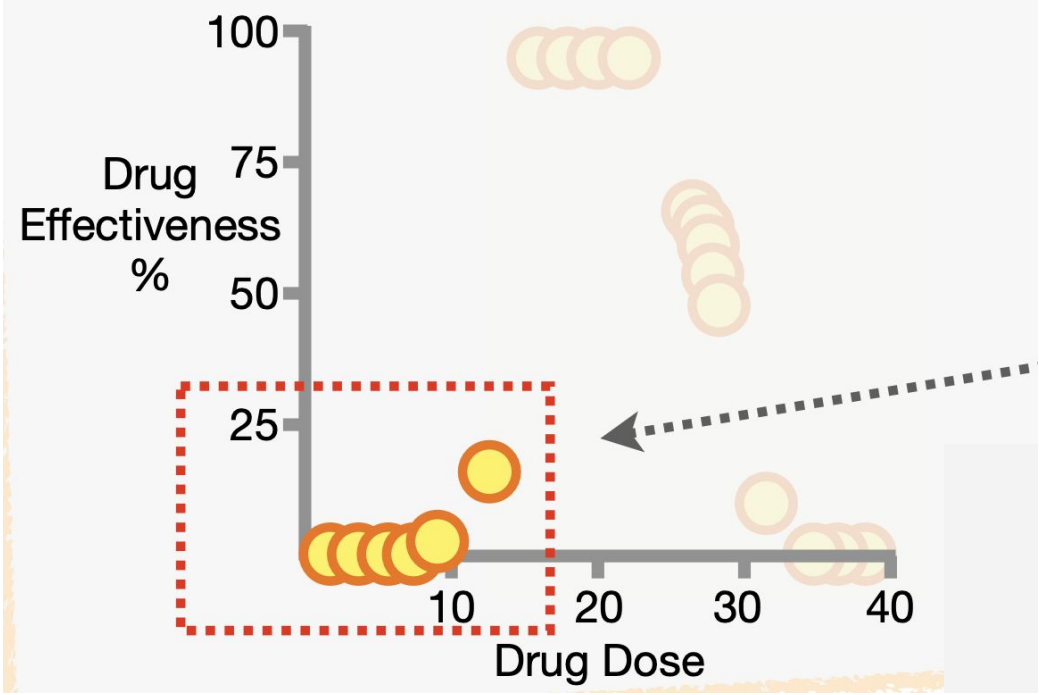


A solution: Regression Tree



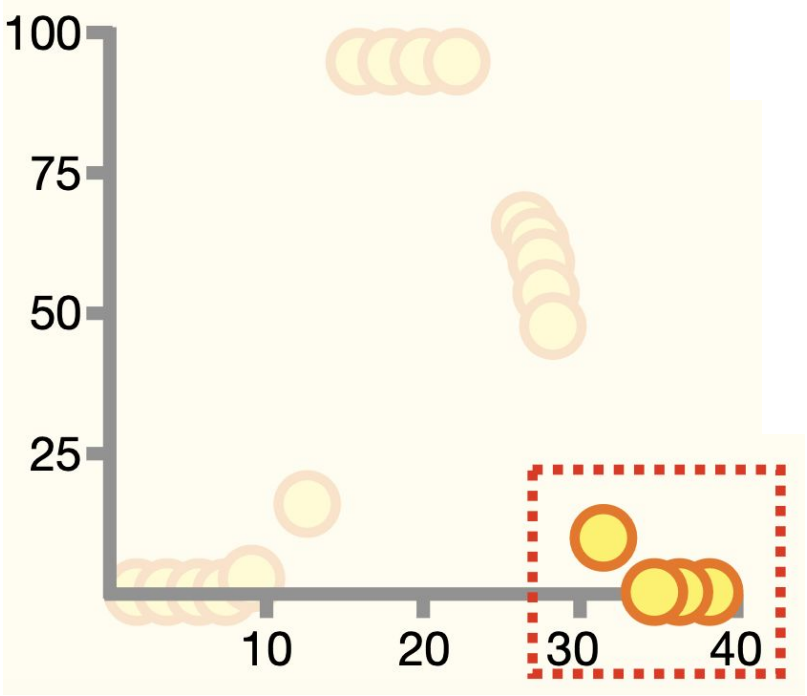
- In this example, if you're given a new Dose and want to know how Effective it will be, you start at the top and ask if the Dose is < 14.5
- If it is, then the drug will be 4.2% Effective. If the Dose is not < 14.5, then ask if the Dose ≥ 29
- If so, then the drug will be 2.5% Effective. If not, ask if the Dose ≥ 23.5
- If so, then the drug will be 52.8% Effective. If not, the drug will be 100% Effective

The Value of Leafs



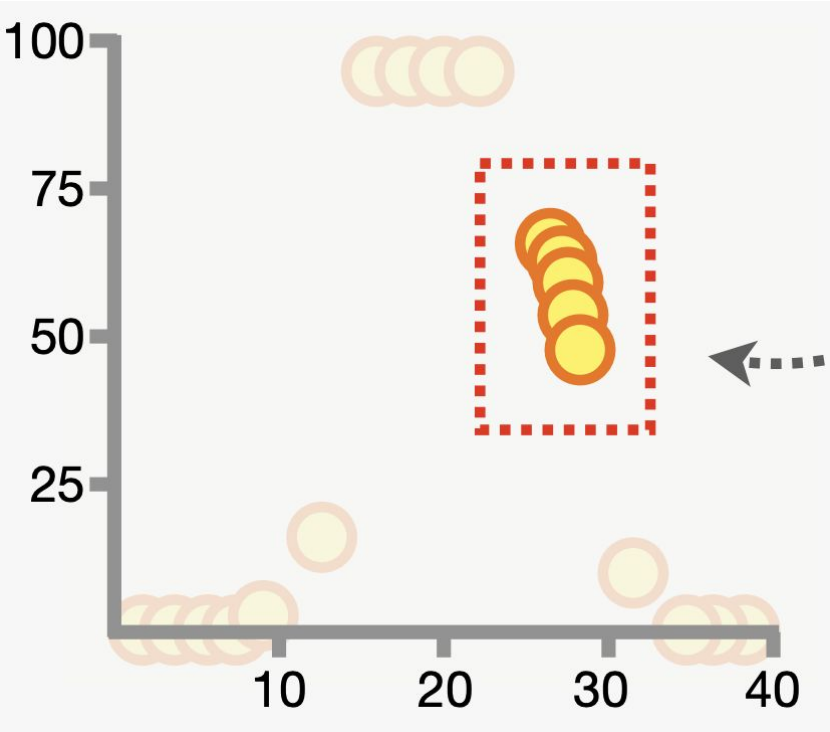
- In this example, the **Regression Tree** makes good predictions because each **Leaf** corresponds to a different cluster of points in the graph. If we have a Dose **< 14.5**, then the output from the **Regression Tree** is the average Effectiveness from these **6** measurements, which is **4.2%**.

The Value of Leafs



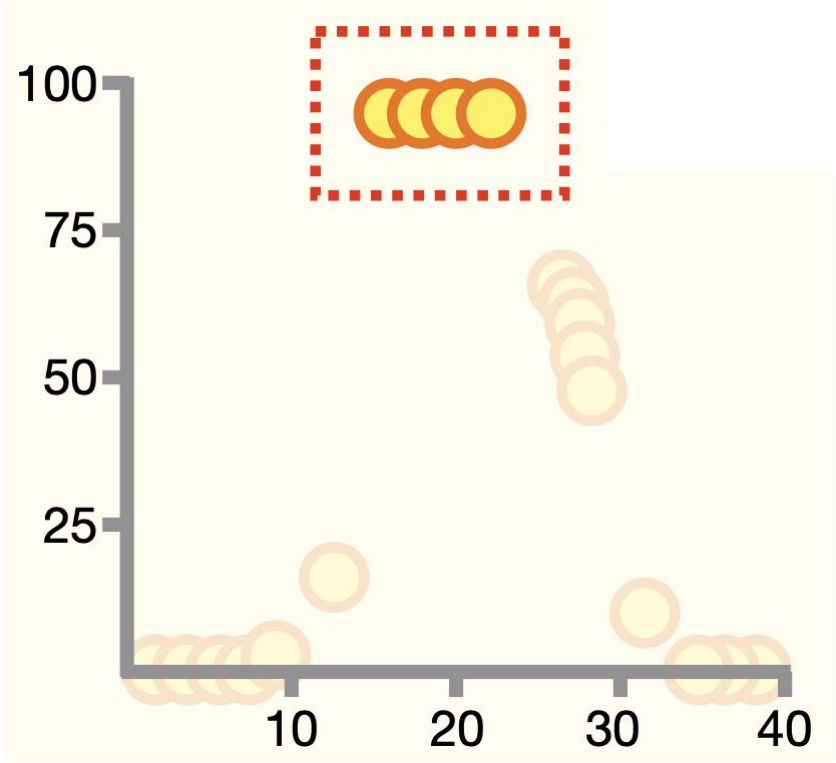
- If the Dose is ≥ **29**, then the output is the average Effectiveness from these **4** measurements, **2.5**%.

The Value of Leafs



- If the Dose is between **23.5** and **29**, then the output is the average Effectiveness from these 5 measurements, **52.8**%.

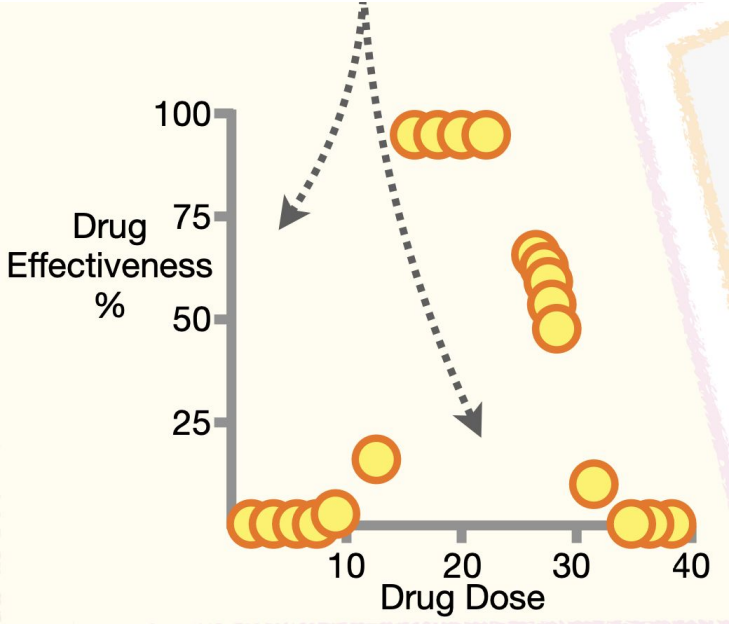
The Value of Leafs



- If the Dose is between **14.5** and **23.5**, then the output is the average Effectiveness from these **4** measurements, **100**%.

Building a Regression Tree

- Given these Training Data, we want to build a Regression Tree that uses Drug Dose to predict Drug Effectiveness

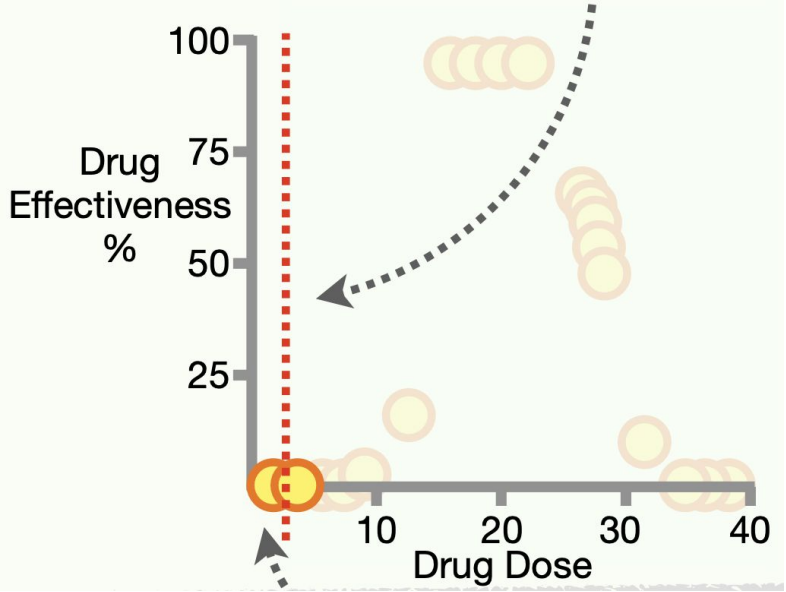


- Just like for Classification Trees, the first thing we do for a Regression Tree is decide what goes in the Root

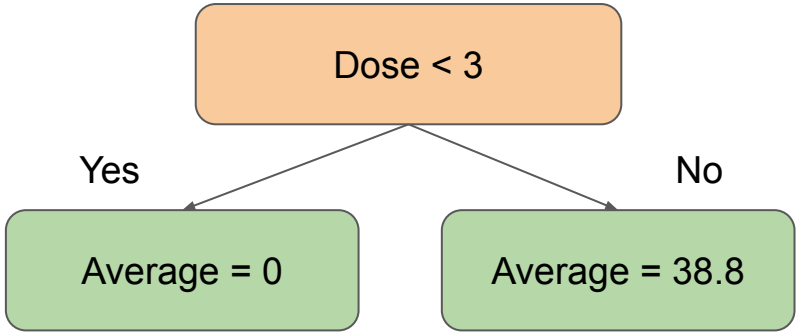
???

Building a Regression Tree

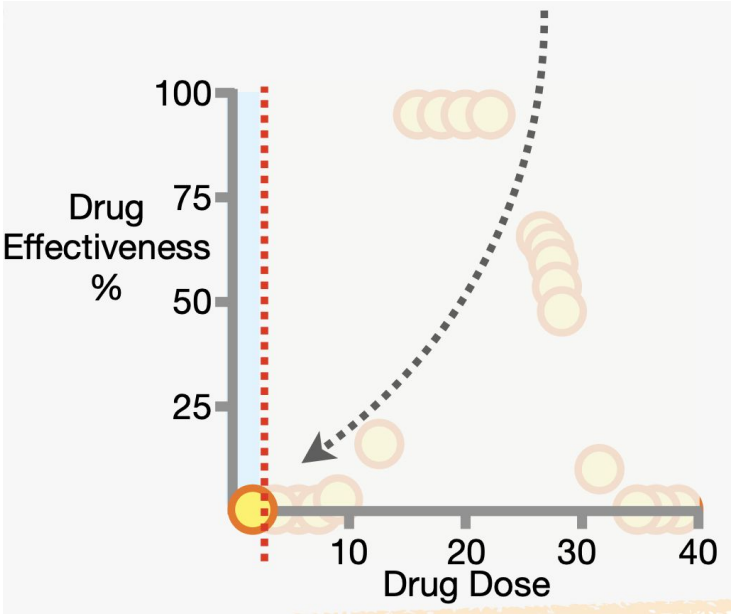
- To make that decision, we calculate the average of the first 2 Doses, which is 3 and corresponds to this dotted line



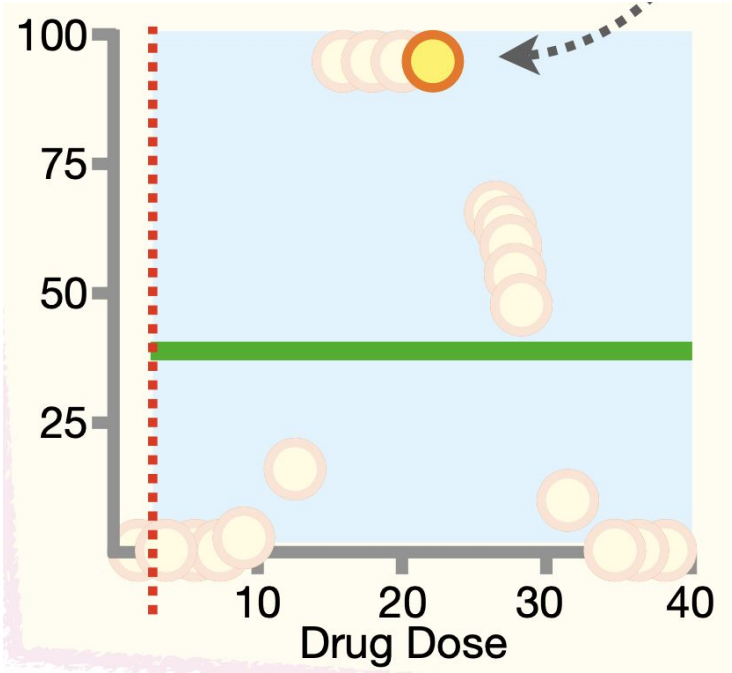
- and then we build a very simple tree that splits the measurements into 2 groups based on whether or not the Dose < 3



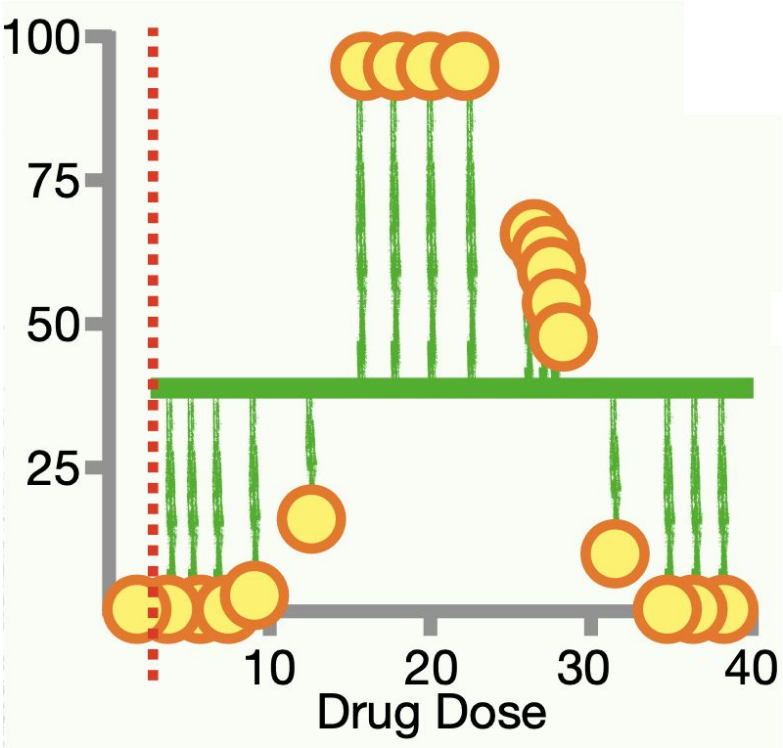
- For the one point with Dose < 3, which has Effectiveness = 0, the Regression Tree makes a pretty good prediction, 0



- For this specific point, which has Dose > 3 and 100% Effectiveness, the tree predicts that the Effectiveness will be 38.8, which is a pretty bad prediction

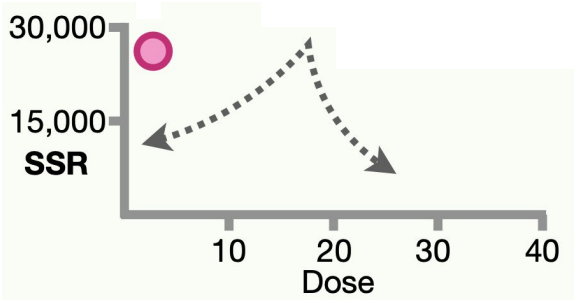


- We can visualize how good or bad the Regression Tree is at making predictions



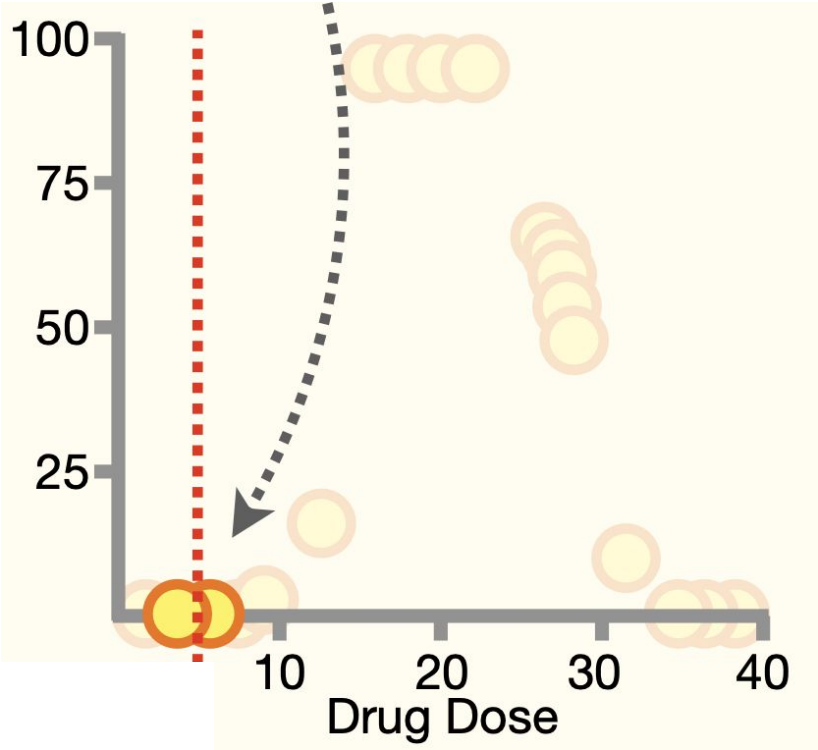
- Quantify how good or bad the predictions by calculating the Sum of the Squared Residuals (SSR)

$$(0 - 0)^2 + (0 - 38.8)^2 + (0 - 38.8)^2 + (0 - 38.8)^2$$
$$+ (5 - 38.8)^2 + (20 - 38.8)^2 + (100 - 38.8)^2$$
$$+ (100 - 38.8)^2 + \ldots + (0 - 38.8)^2$$
$$= 27{,}468.5$$

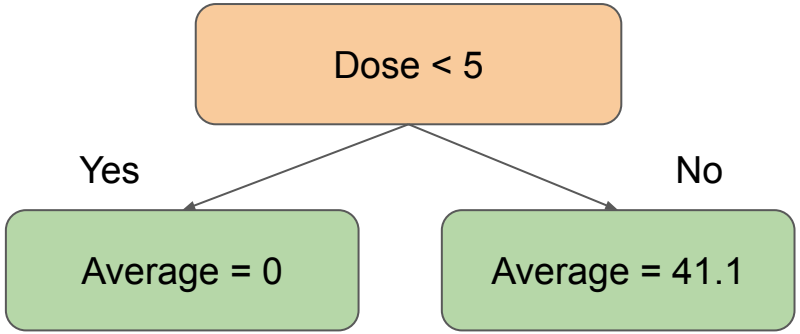


- Lastly, we can compare the SSR for different thresholds by plotting them on this graph, which has Dose on the x-axis and SSR on the y-axis.

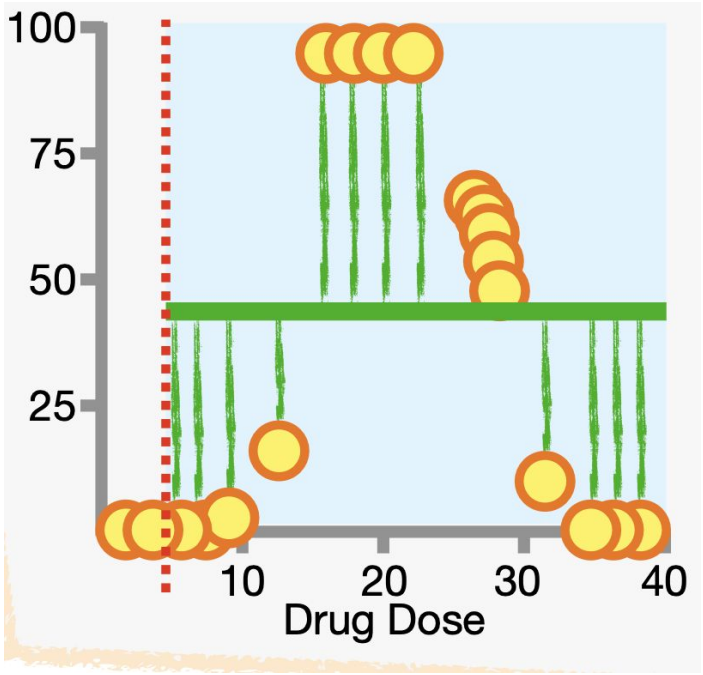
- Now we shift the Dose threshold to be the average of the second and third measurements in the graph, 5



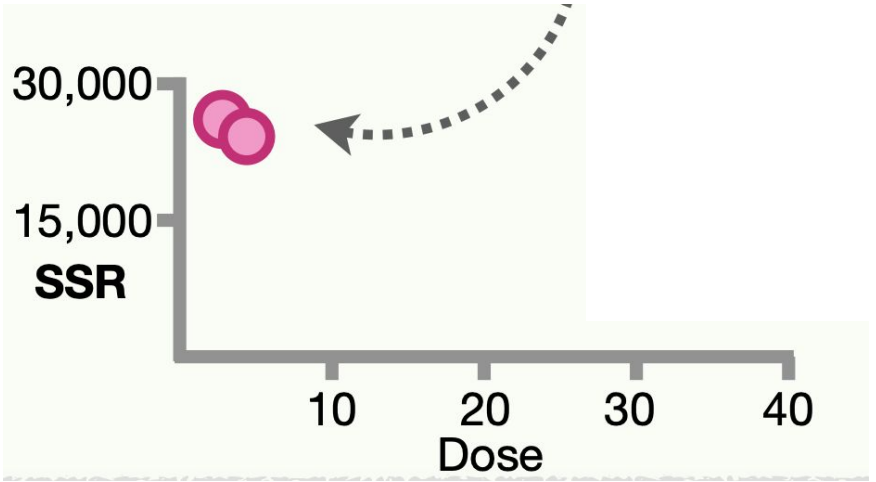
- and we build this super simple tree with Dose < 5 at the Root

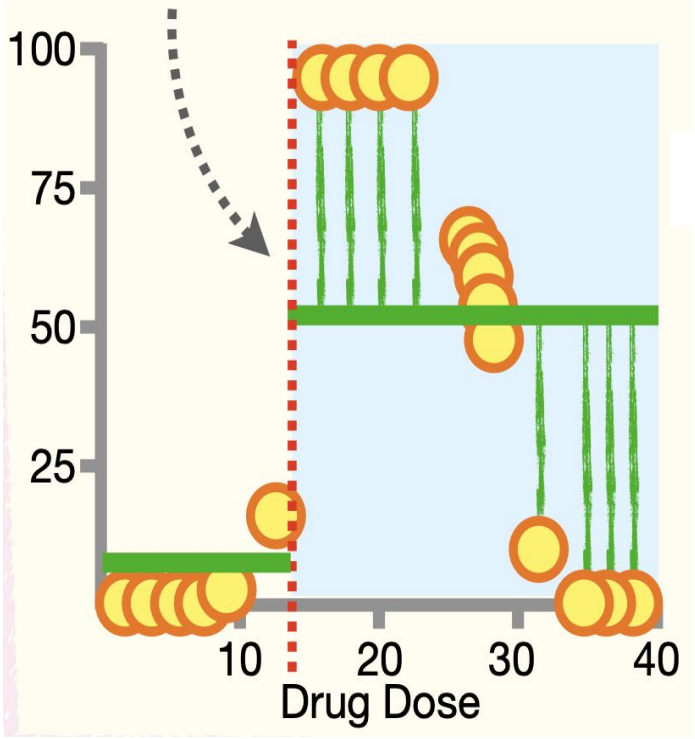


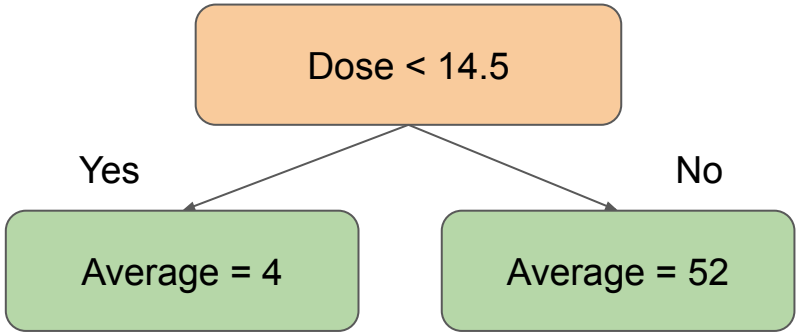
- Now we calculate and plot the SSR for the new threshold, Dose < 5



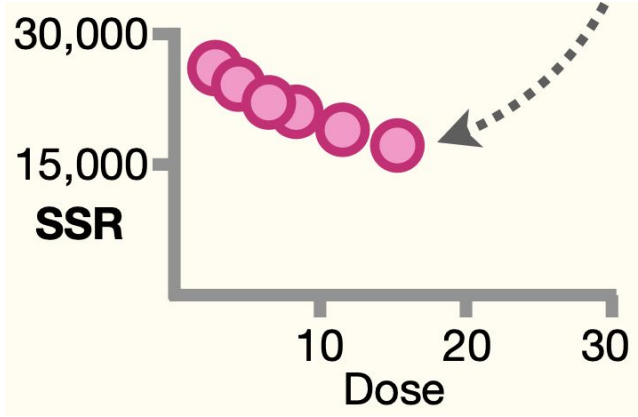
- We see that the SSR for Dose < 5 is less than the SSR for Dose < 3
- Since we're trying to minimize the SSR, Dose < 5 is a better threshold

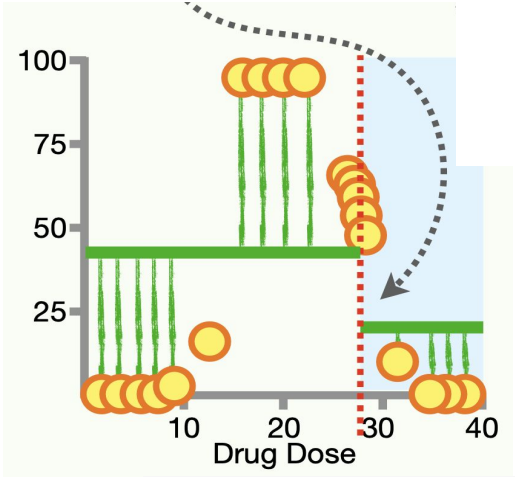


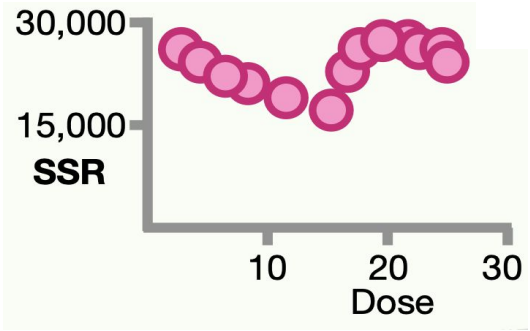




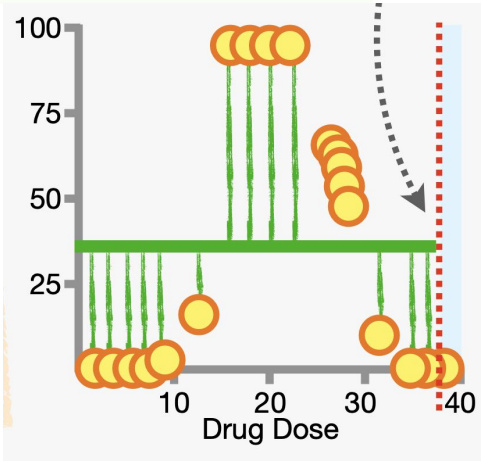
- After shifting the Dose threshold over multiple more steps, the SSR graph looks like this

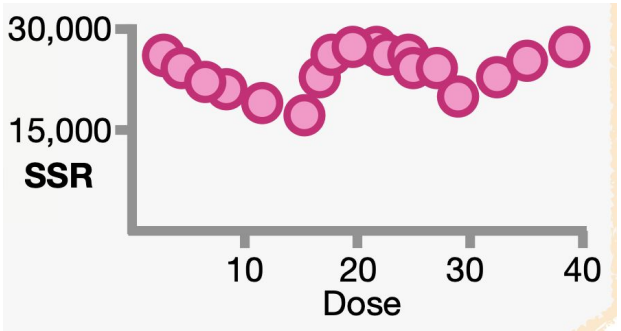






- After shifting the Dose threshold 7 more times, the SSR graph looks like this

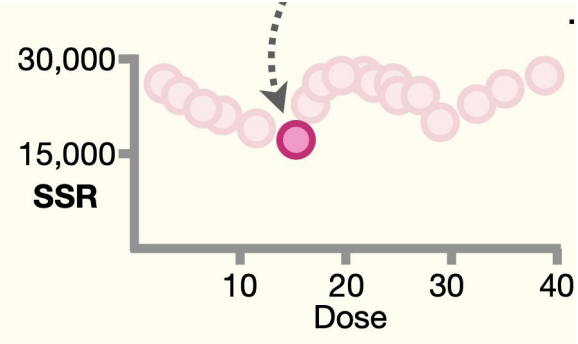


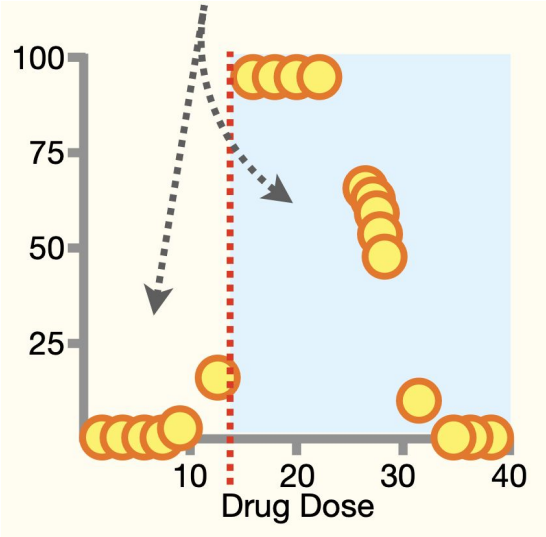


- Finally, after shifting the Dose threshold all the way to the last pair of Doses, the SSR graph looks like this

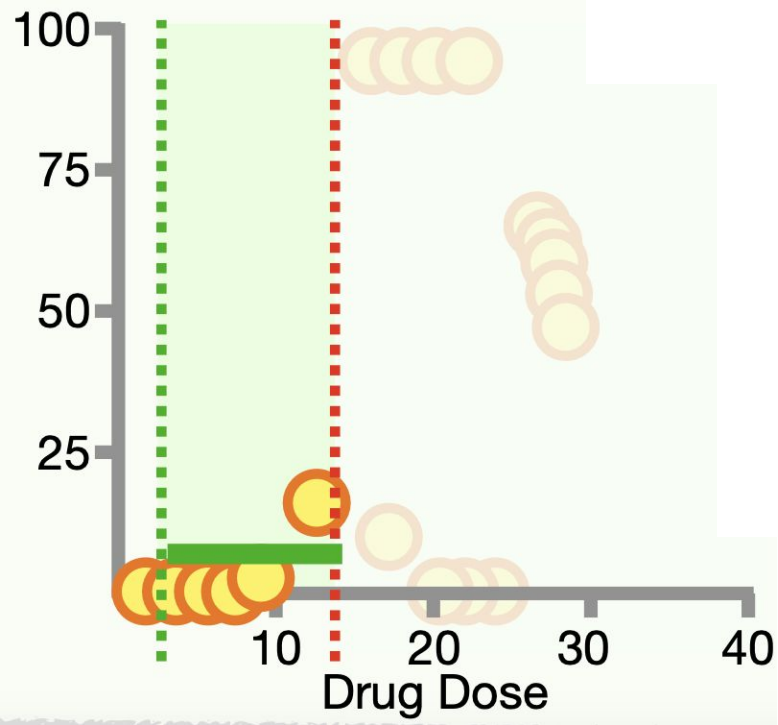
- Looking at the SSRs for each Dose threshold, we see that Dose < 14.5 had the smallest SSR, so Dose < 14.5 will be the Root of the tree, which corresponds to splitting the measurements into two groups based on whether or not the Dose < 14.5

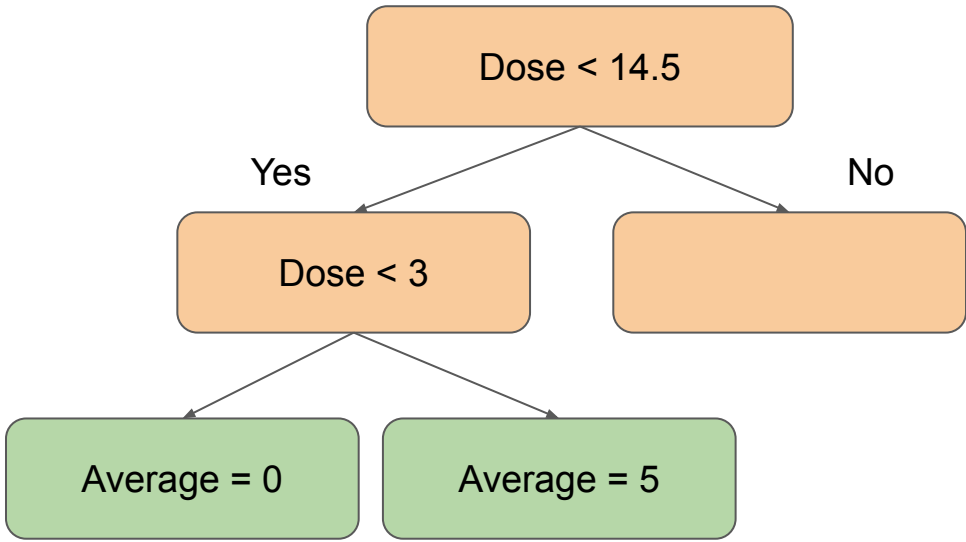
Dose < 14.5

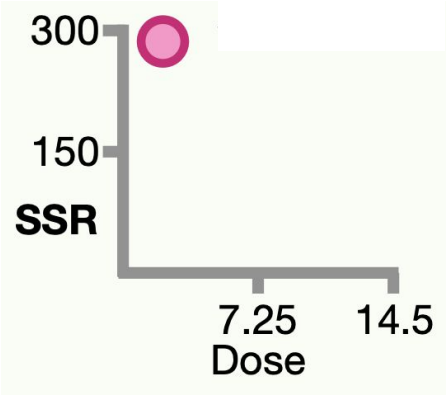




- In other words, just like before, we can average the first two Doses and use that value, 3, as a cutoff for splitting the 6 measurements with Dose < 14.5 into 2 groups

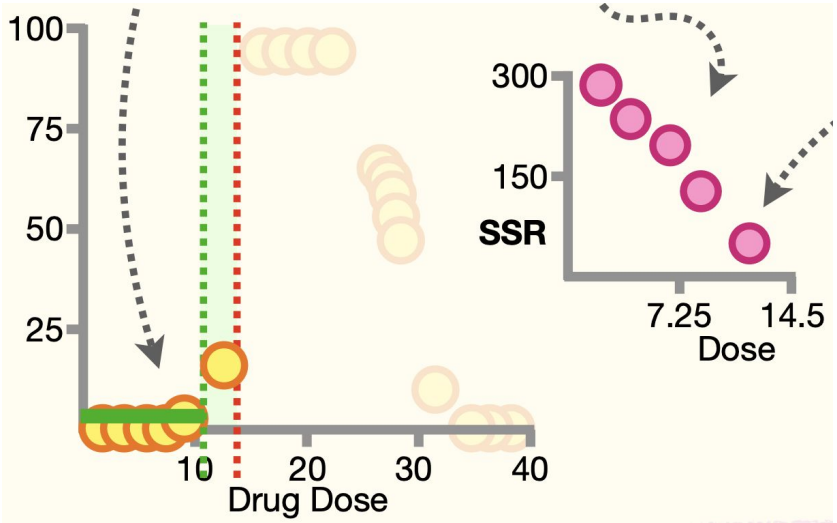




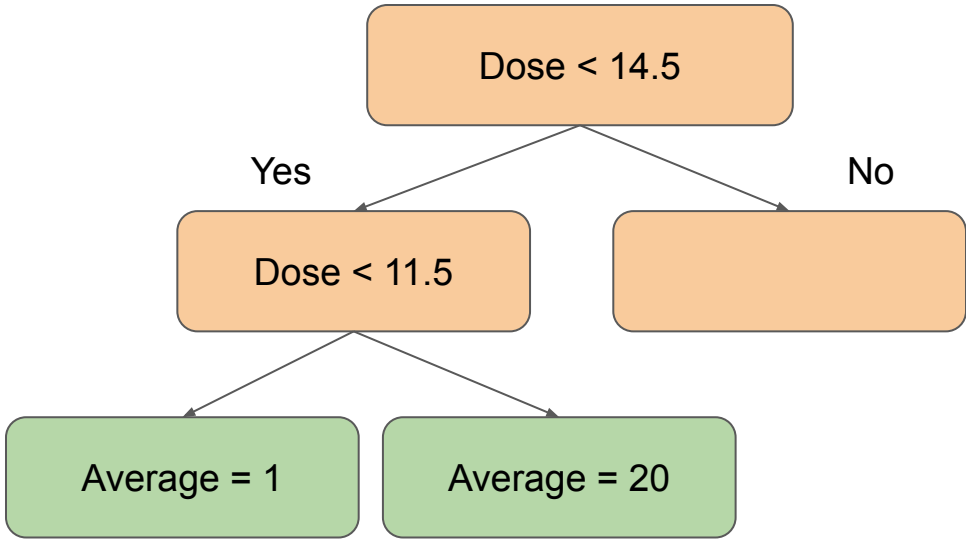


- We calculate the SSR for just those 6 measurements and plot it on a graph

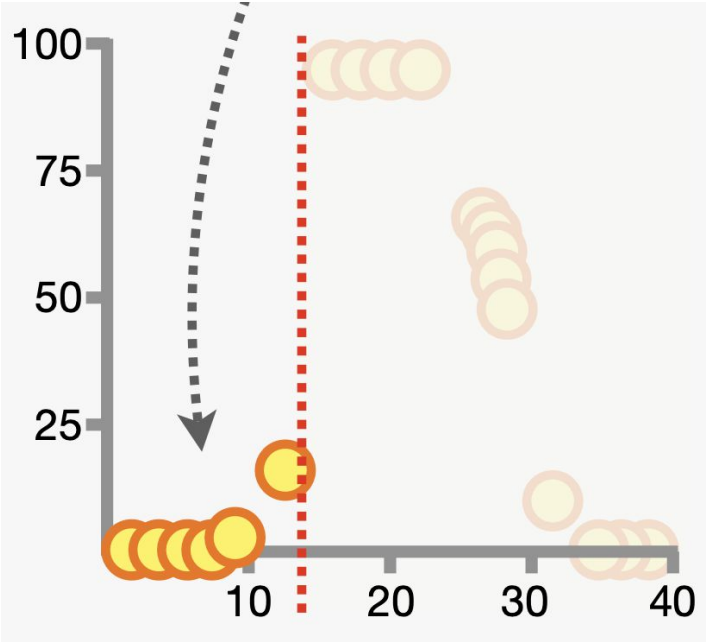
- After calculating the SSR for each threshold for the 6 measurements with Dose < 14.5, we end up with this graph

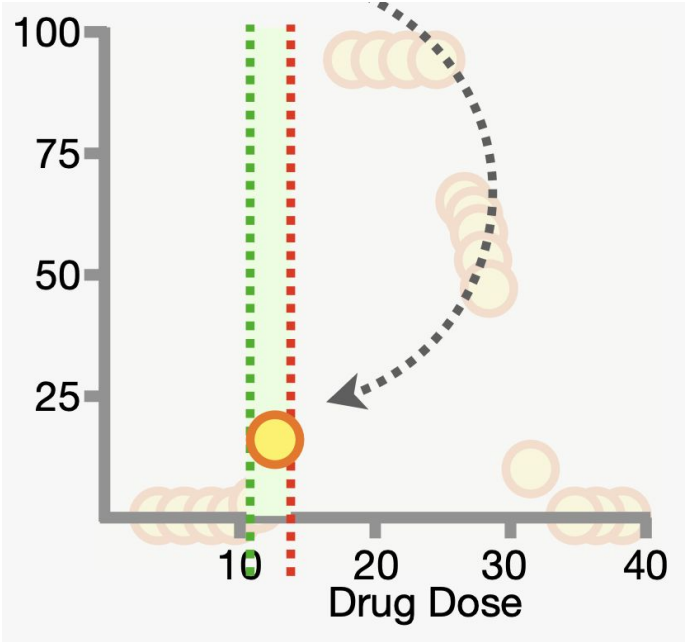


- Then we select the threshold that gives us the lowest SSR, Dose < 11.5, for the next Node in the tree

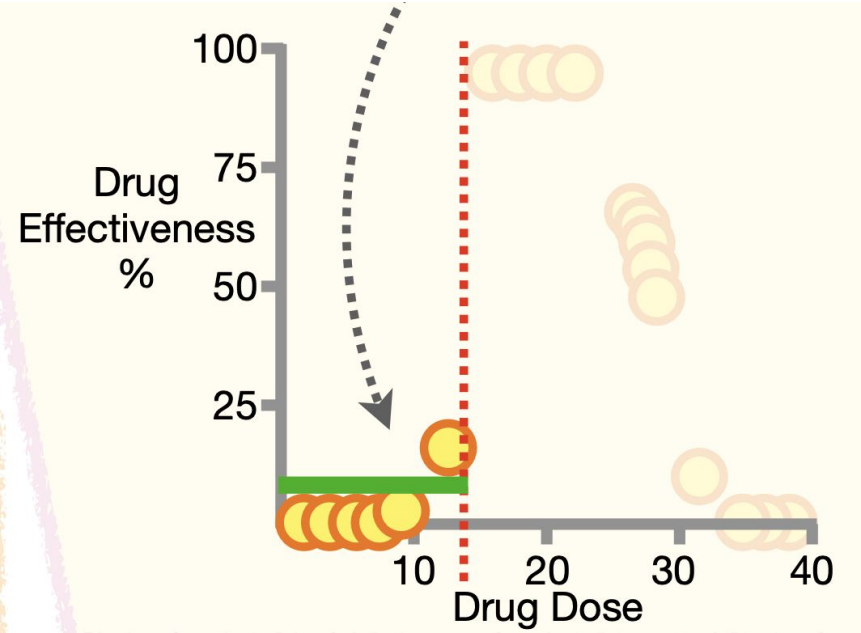


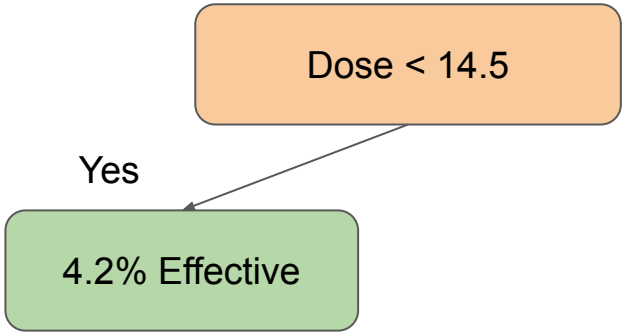
- Earlier, we said in theory we could subdivide the 6 measurements with Dose < **14.5** into smaller groups
- We end up with a single measurement in the **Leaf** on the right
- Making a prediction based on a single measurement suggests that the tree is **Overfit** to the **Training Data** and may not perform well in the future
- The simplest way to prevent this issue is to only split measurements when there are more than some minimum number, which is often **20**. However, since we have so little data in this specific example, we'll set the minimum to **7**

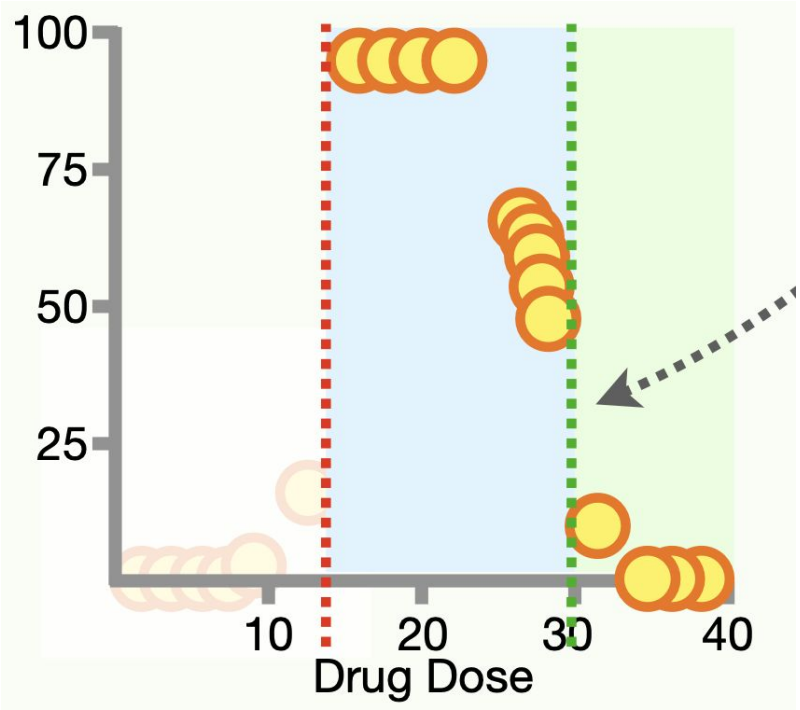




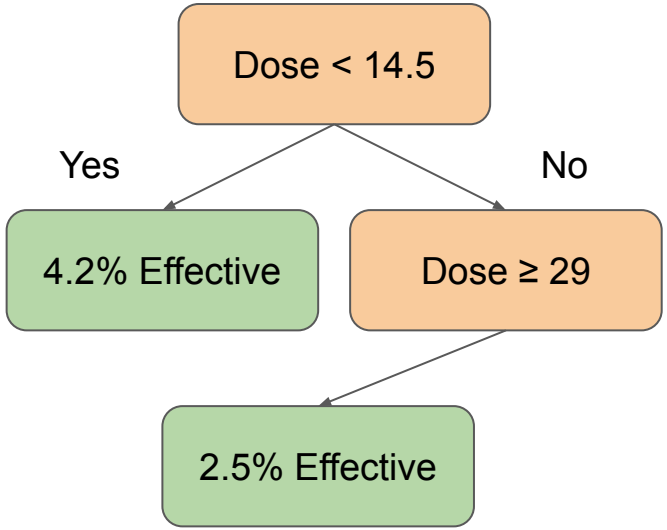
- There are only **6** measurements in the **Node** on the left
- Because we require a minimum of **7** measurements for further subdivision, the **Node** on the left will be a **Leaf**
- The output value for the **Leaf** is the average Effectiveness from the **6** measurements, **4.2**%



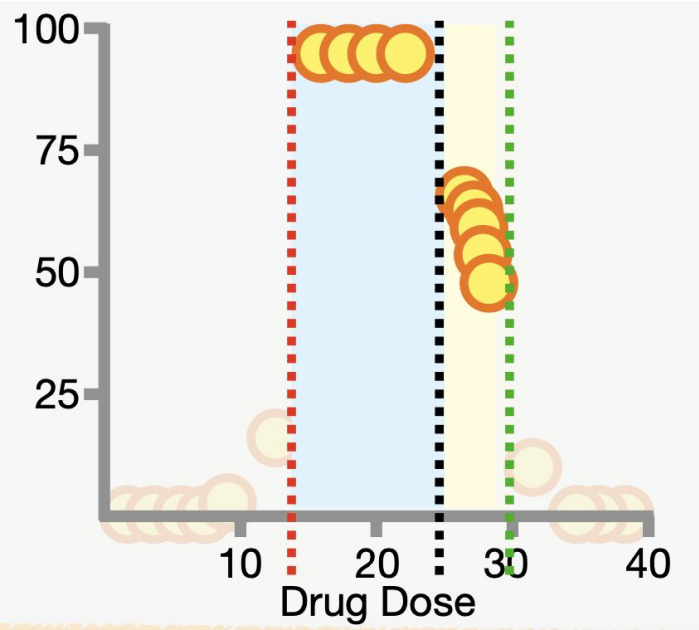


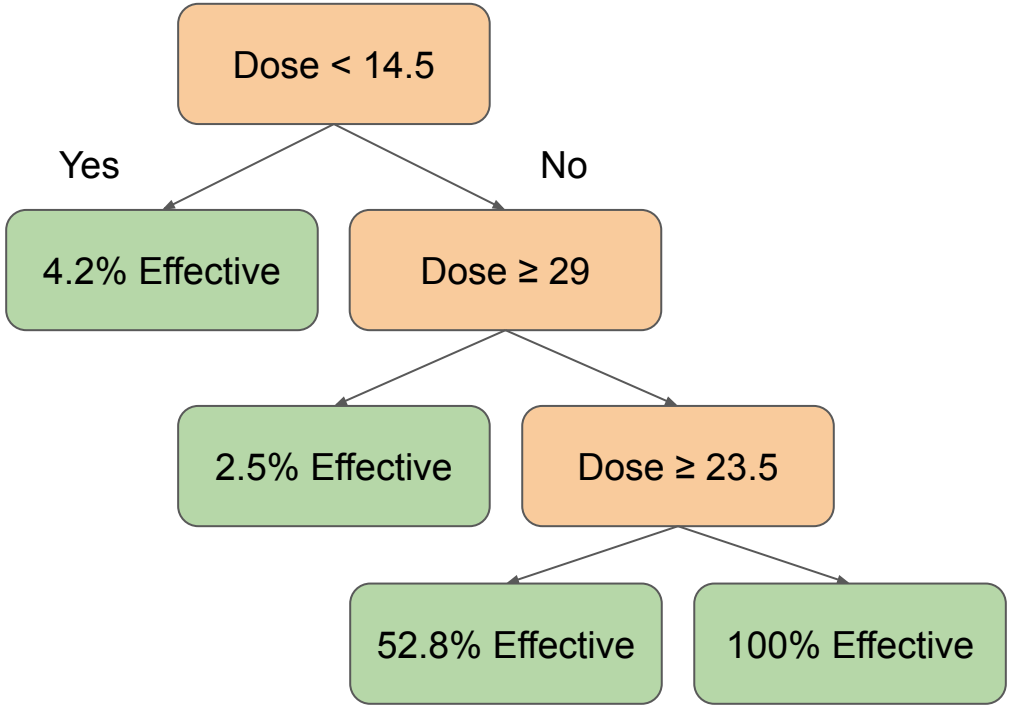


- Since we have more than **7** measurements in the **Node** on the right, we can split them into two groups, and we do this by finding the threshold that gives us the lowest **SSR**
- There are only **4** measurements in the **Node** on the left
- Since the **Node** has fewer than **7** measurements, we'll make it a **Leaf**



- Since there are fewer than **7** measurements in each of these two groups, this will be the last split
- Now, all we have to do is calculate the output values for the last 2 **Leaves**
- At long last, we've finished building the **Regression Tree**



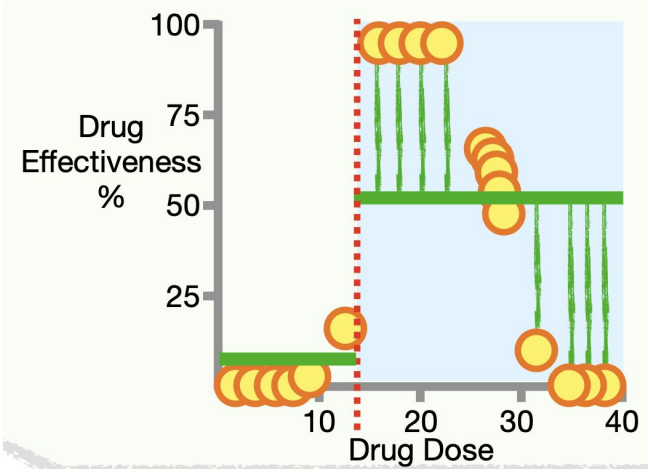


Regression Trees with Multiple Features

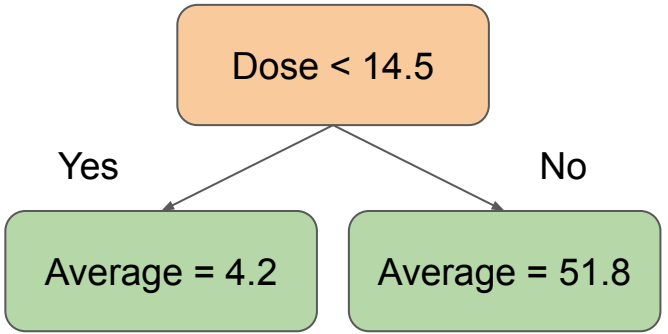
Dose	Age	Sex	Drug Effect
10	25	F	98
20	73	M	0
35	54	F	6
5	12	M	44
etc…	etc…	etc…	etc…

- So far, we've built a **Regression Tree** using a single predictor, Dose, to predict Drug Effectiveness.
- Now let's see how to build a **Regression Tree** to predict Drug Effectiveness using Dose, Age, and Sex

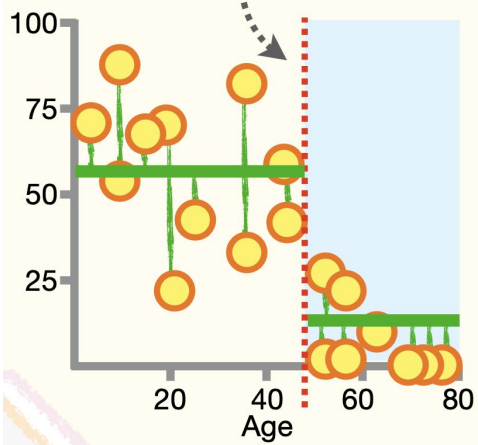
- First, we only use Dose to predict Drug Effectiveness
- Then we select the threshold that gives us the smallest **SSR**
- However, instead of that threshold instantly becoming the **Root**, it only becomes a candidate for the **Root**



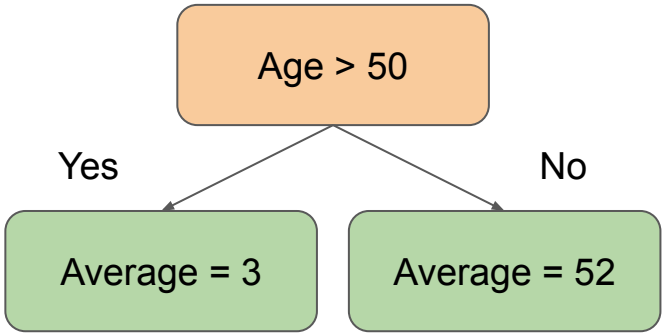
Dose	Age	Sex	Drug Effect
10	25	F	98
20	73	M	0
35	54	F	6
5	12	M	44
etc…	etc…	etc…	etc…



- Then, we ignore Dose and Sex and only use Age to predict Effectiveness
- Select the threshold that gives us the smallest **SSR**
- Get the second candidate for the **Root**

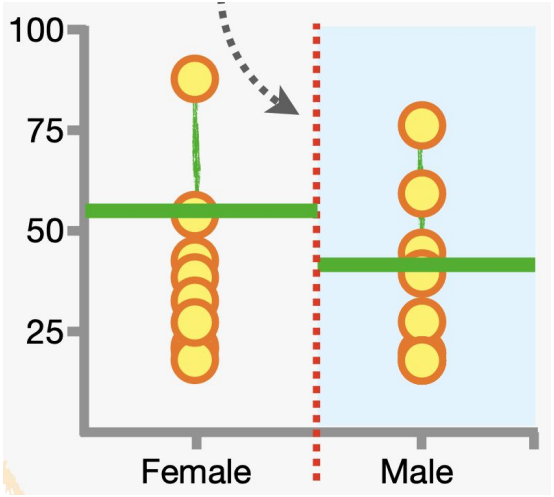


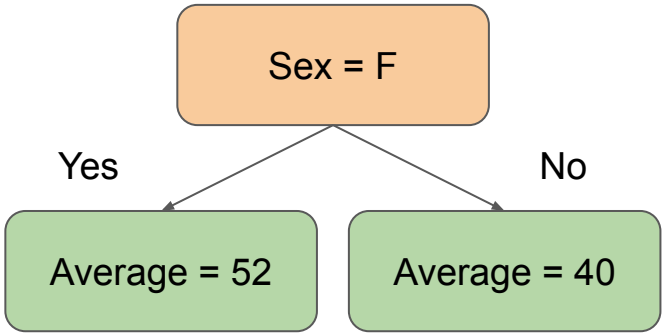
Dose	Age	Sex	Drug Effect
10	25	F	98
20	73	M	0
35	54	F	6
5	12	M	44
etc…	etc…	etc…	etc…

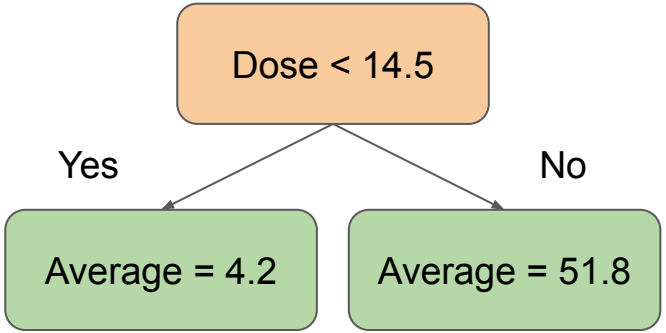


- Lastly, we ignore Dose and Age and only use Sex to predict Effectiveness
- Even though Sex only has one threshold for splitting the data, we still calculate the **SSR**
- Get the third candidate for the **Root**

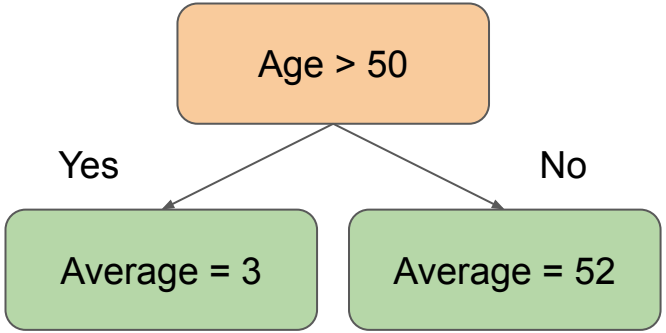
Dose	Age	Sex	Drug Effect
10	25	F	98
20	73	M	0
35	54	F	6
5	12	M	44
etc…	etc…	etc…	etc…



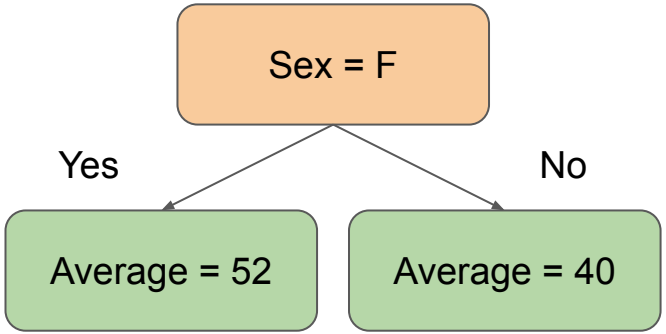




SSR = 19564

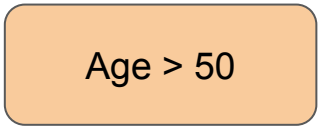


SSR = 12017



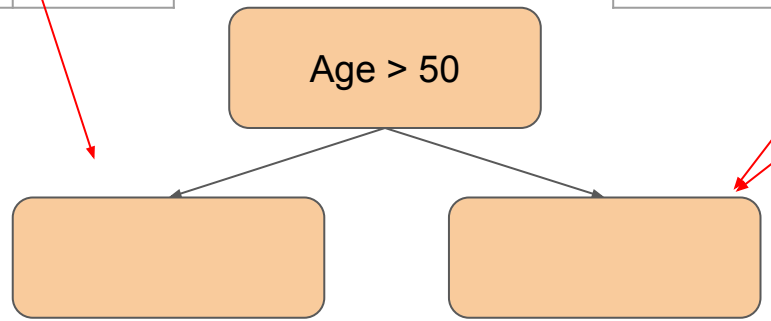
SSR = 20738

- Now we compare the **SSRs** for each candidate for the **Root** and pick the one with the lowest value
- Because Age **> 50** had the lowest **SSR**, it becomes the **Root** of the **Regression Tree**

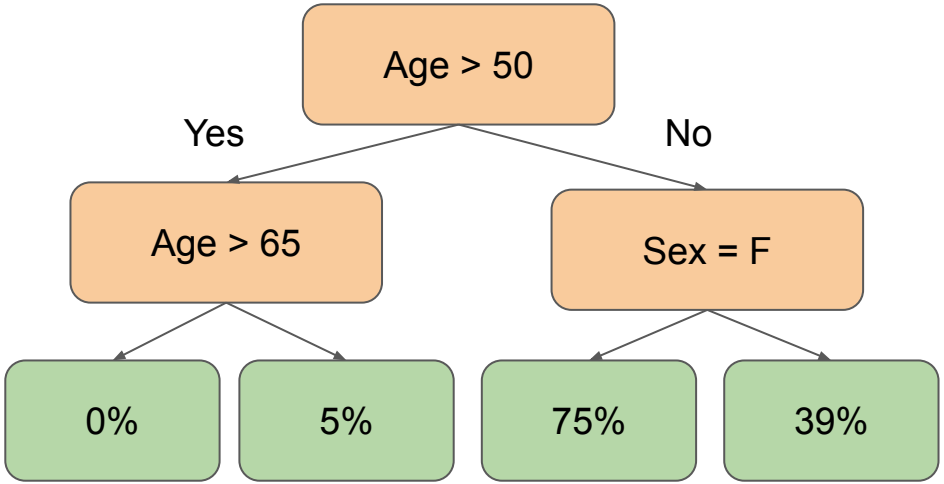


Dose	Age	Sex	Drug Effect
10	25	F	98
20	73	M	0
35	54	F	6
5	12	M	44
etc…	etc…	etc…	etc…

Dose	Age	Sex	Drug Effect
10	25	F	98
20	73	M	0
35	54	F	6
5	12	M	44
etc…	etc…	etc…	etc…

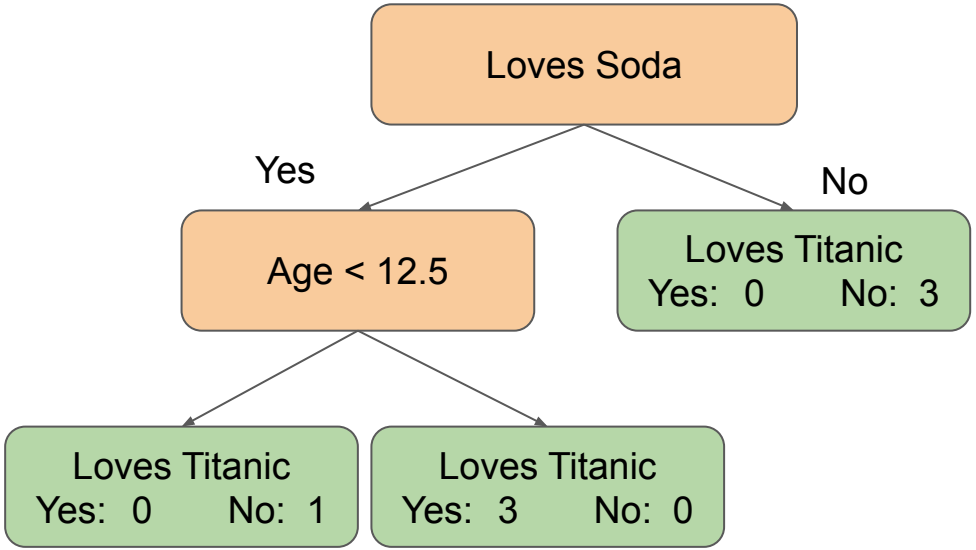


- Then we grow the tree just like before, except now for each split we have **3** candidates, Dose, Age, and Sex, and we select whichever gives us the lowest **SSR**
- Until we can no longer subdivide the data any further. At that point, we're done building the **Regression Tree**



Feature Selection

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



- If some features are never used for split, it means these features are not important for the prediction

Missing Data

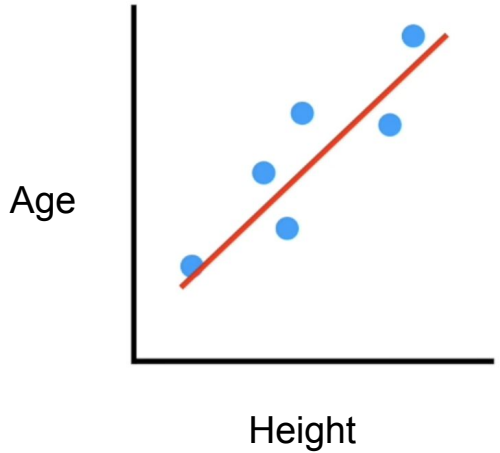
Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	No	7	No
Yes	No	12	No
No	Yes	18	Yes
???	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	Yes	83	No

- We can skip the observation if there is missing data
- Or, we could pick the most common option (“Yes” in this case)
- Or, we could find another column that has the highest correlation with Loves Popcorn and use it as a guide (ex: Loves Soda)

Missing Data

Height	Loves Soda	Age	Loves Titanic
178	No	7	No
167	No	12	No
152	Yes	18	Yes
158	Yes	35	Yes
163	Yes	**???**	Yes
177	No	50	No
172	Yes	83	No

- If the missing value is numerical
- We can replace it with the mean of Age or median
- Or, we could find another column that is correlated with Age (ex: Height), and do a linear regression on the two colmuns



Fine-Tune the Model

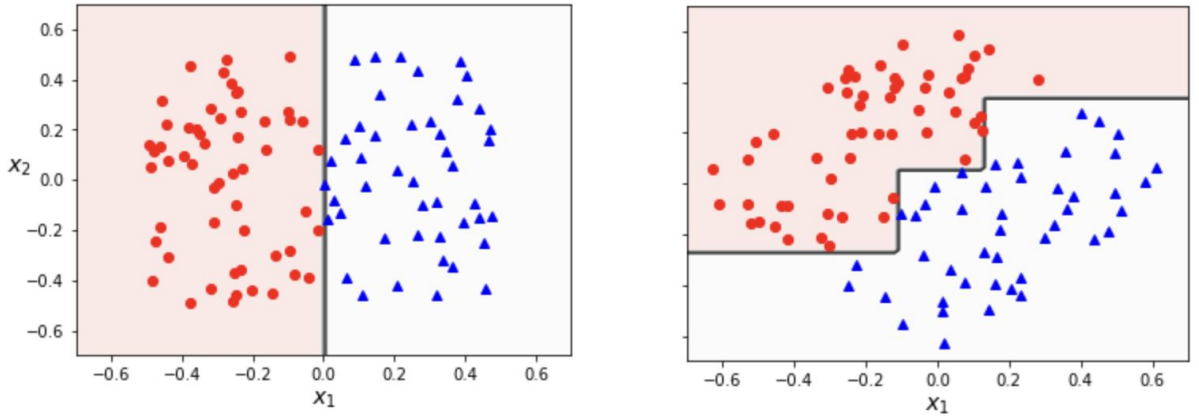
- We need to set rugularization hyperparameters to improve model performance
- How to select the right combinations?
- Grid search!

Pros of Decision Trees

- Finds cut points from the data automatically
  - e.g. no need to add polynomial features by hand
- Will not be affected by feature scale
- Easy to explain when the tree is shallow
  - Each prediction follows a short chain of if-then rules
  - Deep trees and tree ensembles lose much of this clarity

Cons of Decision Trees

- Overfits easily, which leads to high variance
  - Without limits, a tree keeps splitting until it memorizes the training data, so small changes in the data can produce a very different tree
- Sensitive to rotation of the feature space
  - Splits are always parallel to the axes, so a daigonal line boundary becomes a staircase



Cons of Decision Trees

- Splits are chosen greedily, so the final tree may not be the best overall
  - Each split picks what looks best at that step, and earlier splits are never revisited
  - e.g. a split that looks useless now might lead to much better splits later, but the tree never tries it